



Fachhochschule der Wirtschaft
FHDW Bielefeld

Studienarbeit im Modul Classification and Clustering

Kundensegmentierung mit RetailRocket und Online Retail II

Vorgestellt von:

Christian Roth

Senner Straße 68

33647 Bielefeld

`christian.roth@edu.fhdw.de`

Studiengang:

Wirtschaftsinformatik mit Schwerpunkt Data Science (*M.Sc.*)

Prüferin:

Prof. Dr. Yvonne Gorniak

Eingereicht am:

30. September 2025 in Bielefeld

Gendererklärung

Aus Gründen der besseren Lesbarkeit wird auf die gleichzeitige Verwendung der Sprachformen männlich, weiblich und divers (m/w/d) verzichtet. Sämtliche Personenbezeichnungen gelten gleichermaßen für alle Geschlechter.

Abstract

Kunden im E-Commerce verhalten sich sehr unterschiedlich in ihrer Kaufhäufigkeit, ihren Ausgaben und ihrer Reaktion auf Kampagnen. Für Unternehmen liegt darin eine große Chance, denn differenzierte Kundensegmente ermöglichen eine gezieltere Ansprache, nachhaltigere Bindung und wirksamere Sortimentsentscheidungen. Gleichzeitig stellt diese Vielfalt eine Herausforderung dar, da sich Strukturen in den Daten nicht unmittelbar erschließen und klassische Auswertungen oft oberflächlich bleiben.

Diese Arbeit möchte die Forschungsfrage beantworten, welche Segmentstrukturen sich in E-Commerce-Daten verbergen und wie diese so beschrieben werden können, dass Marketing und CRM sie unmittelbar nutzen können. Clustering wird dabei nicht als Selbstzweck verstanden, sondern als Werkzeug, das Datenmuster sichtbar macht und in verständliche Profile übersetzt.

Das Vorgehen orientiert sich am CRISP-DM-Modell. Durch Feature Engineering werden Kennzahlen wie Recency, Frequency und Monetary Value sowie Warenkorb- und Sortimentsmerkmale gebildet. Die Segmentierung erfolgt mit K-Means für kompakte Strukturen und HDBSCAN für dichte-basierte Cluster. Die Qualität wird mit dem Silhouette-Koeffizienten und dem DBCV-Index bewertet. Die Hopkins-Statistik wird ergänzend zur Prüfung der Clustertendenz eingesetzt, ihre Aussagekraft für hochdimensionale, sparse Daten jedoch kritisch eingeordnet.

Die Ergebnisse zeigen mehrere unterscheidbare Segmente, die durch Kennzahlentabellen, Personas und Visualisierungen anschaulich dargestellt werden. Damit können Fachbereiche die Segmente für Kampagnen, Kundenbindungsprogramme und Sortimentsentscheidungen einsetzen. Die Arbeit zeigt, dass sich Segmentstrukturen in E-Commerce-Daten identifizieren, verständlich aufbereiten und in handlungsleitende Marketingmaßnahmen überführen lassen. Ein Ausblick skizziert die Ergänzung durch externe Validierung, Text-Mining und dynamische Segmentierungen.

Inhaltsverzeichnis

Gendererklärung	i
Abstract	ii
Inhaltsverzeichnis	iii
Abbildungsverzeichnis	iv
Tabellenverzeichnis	v
1 Einleitung	6
1.1 Problemstellung	6
1.2 Zielsetzung	6
1.3 Vorgehensweise	7
2 Theoretischer Hintergrund	8
2.1 Kundensegmentierung im E-Commerce	8
2.2 Clustering als Mittel zum Zweck	8
3 Datenbasis und Aufbereitung	10
3.1 Roadmap der Segmentierung im Projekt	10
3.2 Datensätze Online Retail II / RetailRocket	11
4 Methodik (CRISP-DM: Modeling - Theoretische Grundlagen)	12
4.1 Feature Engineering und Preprocessing (RFM, Warenkorbgrößen, Sortimentsmerkmale – konzeptionell)	12
4.2 Clustering-Verfahren (K-Means, HDBSCAN - zentrale Eigenschaften) . . .	14
4.3 Clustertendenz und Modelle Hopkins, K-Means, HDBSCAN	16
4.4 Validierung (Hopkins, Silhouette, DBCV) als Qualitätskontrolle	17
5 Ergebnisse und Diskussion (CRISP-DM: Modeling und Evaluation - praktisch)	19

5.1	Preprocessing (Bereinigung, Skalierung, Variablenableitung – Resultate, Details in Anhang C)	19
5.2	Clustering (K-Means und HDBSCAN – Ergebnisse und Validierung)	19
5.3	Interpretation der Segmente (Clusterprofile, Personas, Visualisierungen) . .	20
6	Diskussion	21
6.1	Vergleich der Verfahren	21
6.2	Implikationen für Marketing und CRM	21
6.3	Grenzen und methodische Limitationen	21
7	Fazit und Ausblick	22
7.1	Beantwortung der Forschungsfrage	22
7.2	Limitationen	22
7.3	Ausblick (z. B. Text Mining, Echtzeit-Segmentierung)	23
	Literaturverzeichnis	24
	Anhang	25
	Anhangsverzeichnis	25
	Anhang A: Prozessmodelle	26
	A.1: Das CRISP-DM-Modell	27
	A.2: Segmentation Management Strategy	28
	Anhang B: Clustering und RFM	31
	B.1: Eindimensionale RFM-Segmentierung	32
	B.2: Mehrdimensionale RFM-Segmentierung	33
	B.3: Retail-Segmentierung mit RFM	33
	B.4: Warenkorb- und Sortimentsmerkmale	37
	Anhang C: Datenaufbereitung und Voranalysen	51
	C.1 Variablenbeschreibung und Quellen	52
	C.2: Abbildung auf ein einheitliches Ereignisschema	56
	C.3: Bereinigungsverfahren	60
	C.4: Normalisierung der Zeitstempel	63
	C.5: Zusammenfassung des bereinigten Datenbestands	67
	C.6 Clustertendenz mit der Hopkins-Statistik	72
	C.7 Modellierungsschritte mit Python: K-Means und HDBSCAN auf Kundenebene	75
	Anhang D: Ergebnisse und Visualisierungen	84

D.1: Clusterprofile	85
D.2: Visualisierungen der Clusterlösungen	87
D.3: Personas der Kundensegmente	93
Glossar	96

Abbildungsverzeichnis

3.1	Eigene Darstellung der Segmentierungs-Roadmap mit Transformation von Rohdaten über Cluster und Segmente hin zu strategischen Maßnahmen . .	11
A.1	Das CRISP-DM-Modell nach Wirth und Hipp (2000), S. 5	27
A.2	Roadmap der Segmentation Management Strategy nach Tsiptsis und Chorianopoulos (2010), S. 213–216	28
A.3	Beispiel für eindimensionale RFM-Einteilungen in fünf Quantile je Dimension. Hervorgehoben ist die RFM-Zelle 351 (Recency = 3, Frequency = 5, Monetary = 1) nach Tsiptsis und Chorianopoulos (2010), S. 336.	32
A.4	Eigene Darstellung eines RFM-Würfels	36
A.5	Warenkorbanalyse verschiedener Kundentypen mit Durchschnittswerten, Bestellhäufigkeit und Vergleichsdarstellung	43
A.6	Sortimentsverhalten: Vergleich von Generalisten (gleichmäßige Diversifikation) und Spezialisten (dominante Kategorie)	50

Tabellenverzeichnis

7.1	Übersicht der Variablen in beiden Datenquellen	55
7.2	Abbildung der Variablen auf das Ereignisschema	56


```

#| eval=TRUE
#| message=FALSE
#| warning=FALSE
#| include=FALSE

# CRAN-Mirror setzen (WICHTIG: Vor allen anderen Operationen)
options(repos = c(CRAN = "https://cran.rstudio.com/"))

# Arbeitsumgebung leeren (alle Objekte entfernen)
rm(list = ls())
gc()

```

Terminal-Output

	used (Mb)	gc trigger (Mb)	max used (Mb)
Ncells	595650 31.9	1339182 71.6	744853 39.8
Vcells	1144443 8.8	8388608 64.0	1878412 14.4

```

# Root für Quarto setzen
knitr::opts_knit$set(root.dir = here::here())

# Installiere und lade erforderliche Bibliotheken
if (!requireNamespace("reticulate", quietly = TRUE))
install.packages("reticulate")
if (!requireNamespace("rmarkdown", quietly = TRUE))
install.packages("rmarkdown")
if (!requireNamespace("shiny", quietly = TRUE)) install.packages("shiny")
if (!requireNamespace("dplyr", quietly = TRUE)) install.packages("dplyr")
if (!requireNamespace("tidyr", quietly = TRUE)) install.packages("tidyr")
if (!requireNamespace("tidyverse", quietly = TRUE))
install.packages("tidyverse")
if (!requireNamespace("forecast", quietly = TRUE))
install.packages("forecast")
if (!requireNamespace("ggpubr", quietly = TRUE)) install.packages("ggpubr")
if (!requireNamespace("viridis", quietly = TRUE)) install.packages("viridis")

```

```
if (!requireNamespace("naniar", quietly = TRUE)) install.packages("naniar")
if (!requireNamespace("GGally", quietly = TRUE)) install.packages("GGally")
if (!requireNamespace("corrplot", quietly = TRUE))
install.packages("corrplot")
if (!requireNamespace("kableExtra", quietly = TRUE))
install.packages("kableExtra")
if (!requireNamespace("gt", quietly = TRUE)) install.packages("gt")
if (!requireNamespace("grid", quietly = TRUE)) install.packages("grid")
if (!requireNamespace("gridExtra", quietly = TRUE))
install.packages("gridExtra")
if (!requireNamespace("patchwork", quietly = TRUE))
install.packages("patchwork")
if (!requireNamespace("mclust", quietly = TRUE)) install.packages("mclust")

if (!requireNamespace("clustertend", quietly = TRUE))
install.packages("clustertend")
if (!requireNamespace("fmsb", quietly = TRUE)) install.packages("fmsb")

library(reticulate)
library(rmarkdown)
library(shiny)
library(dplyr)
library(tidyr)
library(tidyverse)
library(forecast)
library(ggpubr)
library(viridis)
library(naniar)
library(GGally)
library(kableExtra)
library(gt)
library(grid)
library(gridExtra)
```

```
library(patchwork)
library(mclust)
library(clustertend)
library(fmsb)
```

```
#!/ eval=TRUE
#!/ message=FALSE
#!/ warning=FALSE
#!/ include=FALSE

# Erforderliche Bibliotheken installieren (falls nicht vorhanden)
import subprocess
import sys

def install_and_import(package_name, import_name=None):
    """
    Installiert ein Paket und importiert es.
    package_name: Name für pip install
    import_name: Name für import (falls unterschiedlich)
    """
    if import_name is None:
        import_name = package_name

    try:
        __import__(import_name)
    except ImportError:
        print(f"Installing {package_name}...")
        subprocess.check_call([sys.executable, "-m", "pip", "install",
package_name])
    finally:
        globals()[import_name] = __import__(import_name)

# Bibliotheken mit korrekten Namen installieren
packages = [
```

```
("pandas", "pandas"),
("numpy", "numpy"),
("matplotlib", "matplotlib"),
("seaborn", "seaborn"),
("scipy", "scipy"),
("statsmodels", "statsmodels"),
("pulp", "pulp"),
("scikit-learn", "sklearn"), # Hier ist der Unterschied!
("hdbscan", "hdbscan"),
("pyreadr", "pyreadr")
]

for pkg_name, import_name in packages:
    install_and_import(pkg_name, import_name)

# Bibliotheken laden
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from statsmodels.tsa.seasonal import STL
from pulp import LpMaximize, LpProblem, LpVariable, value

# Scikit-learn Imports hinzufügen
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.utils.validation import check_array

# HDBSCAN Import mit Fehlerbehandlung
try:
    import hdbscan
except ImportError:
```

```
print("Installing hdbscan...")
subprocess.check_call([sys.executable, "-m", "pip", "install", "hdbscan"])
import hdbscan

# pyreadr Import
try:
    import pyreadr
except ImportError:
    print("Installing pyreadr...")
    subprocess.check_call([sys.executable, "-m", "pip", "install", "pyreadr"])
    import pyreadr

print("Alle Bibliotheken erfolgreich geladen!")
```

Terminal-Output

Alle Bibliotheken erfolgreich geladen!

1 Einleitung

1.1 Problemstellung

Der Onlinehandel hat sich zu einem der wichtigsten Vertriebskanäle entwickelt. Millionen von Kunden kaufen regelmäßig ein, doch ihre Verhaltensweisen unterscheiden sich erheblich. Manche Kunden tätigen häufig kleine Käufe, andere bestellen selten, geben dabei aber hohe Summen aus. Hinzu kommen Unterschiede in der Sortimentswahl, im Preisbewusstsein und in der Reaktion auf Kampagnen.

Für Unternehmen liegt darin einerseits eine große Chance, da differenzierte Segmente eine gezielte Ansprache und Bindung ermöglichen. Andererseits stellt diese Vielfalt eine Herausforderung dar. Klassische Auswertungen wie Umsatzstatistiken oder Produktlisten zeigen nur an der Oberfläche Unterschiede, erfassen aber nicht die zugrunde liegenden Strukturen. Viele Segmentierungsprojekte scheitern daran, dass sie entweder methodisch zu stark auf Verfahren fokussieren oder dass die Ergebnisse nicht in verständliche, umsetzbare Formate übersetzt werden.

1.2 Zielsetzung

Die Arbeit möchte die Forschungsfrage beantworten, welche Segmentstrukturen sich in E-Commerce-Daten verbergen und wie diese so dargestellt werden können, dass sie für Marketing und CRM unmittelbar nutzbar sind. Mit den Segmentprofilen, den Personas und den daraus abgeleiteten Maßnahmen wird gezeigt, wie Datenmuster in klare Handlungsoptionen übersetzt werden können. Clustering wird dabei als Werkzeug verstanden, nicht als Selbstzweck.

Der Beitrag dieser Arbeit liegt in drei Aspekten. Erstens wird ein nachvollziehbarer

Prozess von der Datenbasis über die Merkmalsbildung bis zur Segmentierung entwickelt. Zweitens werden Segmentstrukturen durch Kennzahlen, Visualisierungen und narrative Personas anschaulich aufbereitet. Drittens werden praxisrelevante Maßnahmen für Marketing und Kundenbindung abgeleitet.

1.3 Vorgehensweise

Das Vorgehen orientiert sich am CRISP-DM-Modell. Zunächst werden theoretische Grundlagen der Kundensegmentierung und die Rolle von Validierungsmaßen erläutert. Anschließend werden die verwendeten Datensätze beschrieben und durch Feature Engineering in aussagekräftige Merkmale transformiert, unter anderem Recency, Frequency, Monetary Value sowie Warenkorb- und Sortimentsvariablen.

Die Segmentierung wird mit zwei Verfahren durchgeführt: K-Means dient als baseline für kompakte Cluster, während HDBSCAN unregelmäßige Strukturen und Rauschen abbilden kann. Die Qualität wird mit Silhouette und DBCV überprüft. Die Hopkins-Statistik wird ergänzend eingesetzt, wobei ihre eingeschränkte Aussagekraft für hochdimensionale Transaktionsdaten kritisch eingeordnet wird.

Im Ergebnisteil werden die Segmente dargestellt, mit Kennzahlen beschrieben und durch Visualisierungen anschaulich gemacht. Personas übersetzen die Ergebnisse in leicht verständliche Profile. Anschließend werden Implikationen für Marketing und CRM diskutiert. Die Arbeit schließt mit einer Reflexion der Ergebnisse, Limitationen und einem Ausblick auf zukünftige Erweiterungen.

2 Theoretischer Hintergrund

2.1 Kundensegmentierung im E-Commerce

Kundensegmentierung ist ein zentrales Instrument im Customer Relationship Management (CRM), um heterogene Kundenmengen in homogene Gruppen zu unterteilen und diese gezielt anzusprechen. Tsipitsis und Chorianopoulos (2010), S. 39 betonen: “Clustering techniques identify meaningful natural groupings of records and group customers into distinct segments with internal cohesion.” Ziel ist die Entwicklung unterscheidbarer Kundentypologien, die „transparent, meaningful, and actionable“ sind (Tsipitsis und Chorianopoulos (2010), S. 129).

Die Bedeutung von Kundensegmentierung im E-Commerce hat mit der Verfügbarkeit umfangreicher Transaktions- und Interaktionsdaten deutlich zugenommen. Personalisierte Angebote, zielgerichtete Kampagnen und differenzierte Kundenbindungsstrategien setzen voraus, dass heterogene Kundenmengen in konsistente Gruppen überführt werden können, die sich anhand ihres Verhaltens und Werts klar unterscheiden.

2.2 Clustering als Mittel zum Zweck

In dieser Arbeit wird Clustering nicht als Selbstzweck betrachtet, sondern als Werkzeug, um verborgene Muster zu erkennen und in verständliche Kundensegmente zu übersetzen. Zwei Verfahren stehen dabei im Mittelpunkt: K-Means und HDBSCAN.

K-Means basiert auf der Idee, die Varianz innerhalb von Clustern iterativ zu reduzieren. Es „starts with an initial cluster solution which is updated and adjusted until no further refinement is possible ... Each iteration refines the solution by reducing the within-cluster variation“ (Tsipitsis und Chorianopoulos (2010), S. 85). Jedes Cluster wird dabei durch sein

Zentrum (Zentroid) repräsentiert, und die Anzahl der Cluster muss vorab festgelegt werden. Dieses Verfahren eignet sich besonders für kompakte, globuläre Strukturen.

HDBSCAN hingegen ist eine Weiterentwicklung dichtebasierter Verfahren. Dichtebasierte Ansätze identifizieren Gruppen in Regionen hoher Punktdichte und weisen spärlich besetzte Bereiche als Rauschen aus; DBSCAN nutzt dafür die Parameter ε und MinPts (Chakraborty et al. (2022), S. 87–88). Damit lassen sich auch nicht-konvexe Formen abbilden und Ausreißer explizit behandeln. K-Means eignet sich dagegen vor allem für kompakte, annähernd kugelförmige Strukturen, während hierarchische bzw. dichtebasierte Verfahren arbiträre Formen robuster erfassen können (Chakraborty et al. (2022), S. 96). HDBSCAN kombiniert diese Eigenschaften, indem es auf einer hierarchischen Betrachtung lokaler Dichten aufbaut und dadurch stabilere Cluster extrahieren kann.

Der gesamte Prozess wird im Rahmen von CRISP-DM strukturiert. Wirth und Hipp (2000), S. 4 beschreiben: „The life cycle of a data mining project is broken down in six phases“, die von Business Understanding über Data Understanding und Data Preparation bis zu Modeling, Evaluation und Deployment reichen. Diese Phasen sind nicht linear, sondern hochgradig iterativ: “it is never the case that a phase is completely done before the subsequent phase starts” (Wirth und Hipp (2000), S. 9). Eine ausführliche Darstellung und die konkrete Anwendung auf diese Arbeit finden sich in A.1: CRISP-DM-Modell.

3 Datenbasis und Aufbereitung

3.1 Roadmap der Segmentierung im Projekt

Die Abbildung Abbildung 3.1 veranschaulicht den konzeptionellen Ablauf der Segmentierung im Rahmen dieser Studie. Ausgangspunkt sind zahlreiche einzelne Beobachtungen, die in den Datensätzen Online Retail II und RetailRocket vorliegen. Diese Vielzahl an Transaktionen, Warenkörben und Interaktionen erscheint zunächst unstrukturiert und komplex.

Im nächsten Schritt werden diese Daten mithilfe von Clustering-Verfahren wie K-Means und HDBSCAN in Gruppen überführt, die gemeinsame Muster erkennen lassen. Die entstehenden Cluster verdichten sich zu interpretierbaren Segmenten, die durch Kennzahlen, Profile und Personas beschrieben werden können. Dieser Übergang macht sichtbar, dass Segmentierung nicht nur eine rechnerische Klassifikation ist, sondern eine inhaltliche Reduktion von Datenkomplexität in klar unterscheidbare Kundengruppen.

Der abschließende Teil der Abbildung symbolisiert die strategische Nutzung dieser Segmente. Auf Basis der erarbeiteten Kundengruppen können konkrete Maßnahmen für Marketing und CRM entwickelt werden. Damit endet der Prozess nicht bei der Modellierung, sondern wird in eine übergeordnete Strategie überführt, die operative Entscheidungen und Kampagnen unterstützt. Die Visualisierung verdeutlicht somit die Transformation von Rohdaten über Cluster und Segmente hin zu strategischen Maßnahmen und bildet den roten Faden für den weiteren praktischen Teil der Arbeit.

Die dargestellte Roadmap knüpft an das in A.2: Segmentation Management Strategy erläuterte Referenzmodell an. Sie macht deutlich, dass der Segmentierungsprozess als Abfolge von Transformationen zu verstehen ist, die von der Datenbasis über die Modellierung bis zur strategischen Umsetzung führen.

Im Anschluss an die Beschreibung der Datensätze erfolgt die Merkmalsbildung durch

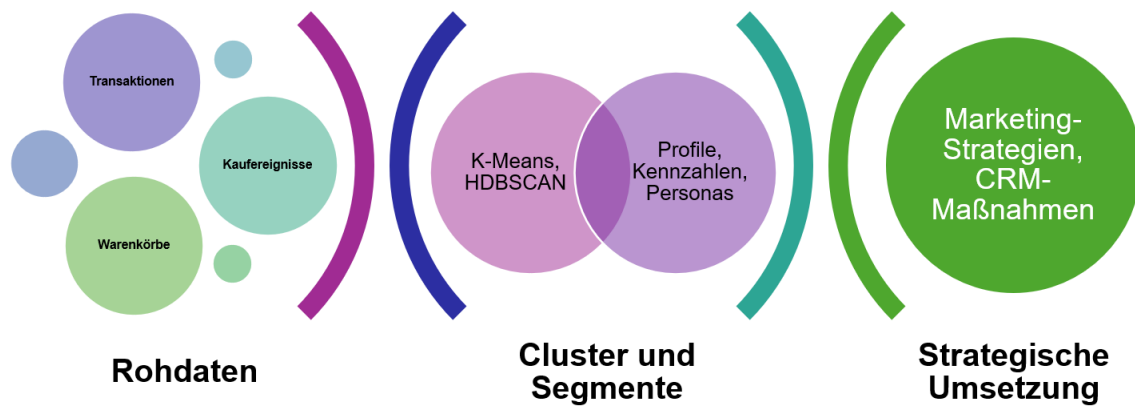


Abbildung 3.1: Eigene Darstellung der Segmentierungs-Roadmap mit Transformation von Rohdaten über Cluster und Segmente hin zu strategischen Maßnahmen

Feature Engineering. Dabei werden aus den Rohdaten zentrale Kennzahlen wie Recency, Frequency, Monetary Value sowie warenkorb- und sortimentsbezogene Variablen abgeleitet. Erst durch diese Konstrukte wird es möglich, im nächsten Schritt Clustermodelle anzuwenden.

Schließlich wird im dritten Teil die Datenaufbereitung behandelt, die für die Robustheit der Modelle entscheidend ist. Dazu gehören die Bereinigung von Ausreißern, die Handhabung fehlender Werte sowie die Skalierung der Merkmale. Mit diesen vorbereitenden Schritten wird die Basis geschaffen, auf der die in der Roadmap dargestellte Transformation technisch und analytisch realisiert werden kann.

3.2 Datensätze Online Retail II / RetailRocket

Um die Forschungsfrage zu beantworten, welche Segmentstrukturen sich in E-Commerce-Daten verbergen und wie diese darstellbar sind, werden zwei komplementäre Datensätze genutzt. RetailRocket erfasst ereignisbasierte Interaktionen wie Ansichten, Warenkorbaktionen und Transaktionen (Zykov et al., 2022). Online Retail II hingegen stellt transaktionsbasierte Rechnungsdaten mit Artikelnummern, Mengen, Preisen und Zeitstempeln bereit (*Datasets - UCI Machine Learning Repository*, 2019).

Die Kombination beider Perspektiven erlaubt es, sowohl verhaltensnahe Muster (RetailRocket) als auch wertorientierte Strukturen (Online Retail II) zu identifizieren. Damit wird die Grundlage gelegt, um robuste und vielseitige Segmentstrukturen zu erschließen.

4 Methodik (CRISP-DM: Modeling - Theoretische Grundlagen)

4.1 Feature Engineering und Preprocessing (RFM, Warenkorbgrößen, Sortimentsmerkmale – konzeptionell)

Damit Segmentstrukturen in den Daten sichtbar werden, müssen aus den Rohdaten aussagekräftige Merkmale abgeleitet werden. Dieser Schritt wird als Feature Engineering bezeichnet (Tsiptsis und Chorianopoulos, 2010, S. 337).

Im Zentrum stehen die klassischen RFM-Variablen, die drei grundlegende Dimensionen des Kaufverhaltens beschreiben:

- *Recency* erfasst den zeitlichen Abstand seit dem letzten Kauf,
- *Frequency* gibt die Anzahl der Käufe in einem bestimmten Zeitraum wieder,
- *Monetary* misst den Umsatz im selben Zeitraum.

Durch die Kombination dieser Dimensionen können Kunden in Zellen eingeteilt werden, die Verhalten und Wert systematisch abbilden. So steht etwa die Zelle 555 für besonders wertvolle Kunden (jüngster Kauf, hohe Frequenz, hoher Umsatz), während 151 Kunden mit langer Kaufpause, geringer Frequenz und geringem Umsatz beschreibt (Tsiptsis und Chorianopoulos, 2010, S. 334–338). Diese Zellenlogik ist leicht verständlich, skaliert aber schlecht, da mit Quintilen bis zu 125 Klassen entstehen können.

Um diese Komplexität zu reduzieren, lässt sich RFM in eine kontinuierliche Kennzahl überführen. Dabei werden die drei Dimensionen zu einem gewichteten Summenscore zusammengefasst:

$$\text{RFM-Score} = w_R \cdot R + w_F \cdot F + w_M \cdot M, \quad w_R + w_F + w_M = 1.$$

Der Vorteil dieser Verdichtung liegt in der besseren Vergleichbarkeit: Statt einer Vielzahl von RFM-Zellen entsteht eine eindimensionale Metrik, die sich leicht sortieren, gruppieren und auch als Input für Clusterverfahren verwenden lässt.

Die Gewichtung der Parameter ist flexibel gestaltbar. Unternehmen können etwa den zeitlichen Abstand (Recency) stärker gewichten, wenn Reaktivierungskampagnen im Vordergrund stehen, oder die Häufigkeit (Frequency), wenn Loyalität im Fokus liegt. Durch Variation der Gewichte w_R , w_F und w_M wird unmittelbar sichtbar, wie sich die Kundenklassifikation verschiebt (Tsipitsis und Chorianopoulos, 2010, S. 338).

Eine anschauliche Visualisierung der RFM-Zellen und ihrer logischen Struktur findet sich in Anhang B.3: Retail-Segmentierung mit RFM.

Ergänzend zur klassischen RFM-Analyse werden in der Literatur weitere behaviorale Variablen vorgeschlagen, um das Kundenverhalten differenzierter zu erfassen. Besonders relevant sind *Warenkorbgrößen* und *Sortimentsmerkmale* (Tsipitsis und Chorianopoulos, 2010, S. 334).

Warenkorbgrößen (*Basket Size*) beschreiben, wie umfangreich einzelne Bestellungen typischerweise ausfallen. Sie können entweder über die durchschnittliche Anzahl enthaltener Artikel oder über den durchschnittlichen Bestellwert erfasst werden. Formal lässt sich die mittlere Warenkorbgröße als Quotient aus der Gesamtzahl der Artikel eines Kunden und der Anzahl seiner Bestellungen ausdrücken:

$$\text{Average Basket Size} = \frac{\text{Gesamtzahl Artikel eines Kunden}}{\text{Anzahl seiner Bestellungen}}.$$

Sortimentsmerkmale (*Relative Spending per Product Group*) erfassen die Breite beziehungsweise die Fokussierung der Ausgaben eines Kunden. Sie geben an, welcher Anteil des Gesamtumsatzes auf einzelne Produktgruppen entfällt. Formal kann der Anteil einer Kategorie i als Verhältnis von Umsatz in dieser Kategorie zum Gesamtumsatz des Kunden dargestellt werden:

$$\text{Sortimentsanteil}_i = \frac{\text{Umsatz in Kategorie } i}{\text{Gesamtumsatz des Kunden}}.$$

Ein hoher Wert in einer Kategorie kennzeichnet einen „Spezialisten“, während eine gleichmäßigere Verteilung auf viele Kategorien auf einen „Generalisten“ hinweist.

Eine ausführlichere und beispielhafte Darstellung dieser Konzepte erfolgt in Anhang B.4: Warenkorb- und Sortimentsmerkmale, wo anhand von Illustrationen unterschiedliche Kundentypen wie „Frequent Shopper“, „Bulk Buyer“, „Generalisten“ und „Spezialisten“ gegenübergestellt werden.

4.2 Clustering-Verfahren (K-Means, HDBSCAN - zentrale Eigenschaften)

In dieser Arbeit werden zwei komplementäre Verfahren eingesetzt, um unterschiedliche Strukturannahmen im Merkmalsraum abzudecken: **K-Means** als zentroidbasiertes Referenzverfahren sowie **HDBSCAN** als dichtebasierte, hierarchische Methode. Die Kombination erlaubt sowohl kompakte, annähernd kugelförmige Muster (K-Means) als auch unregelmäßige Strukturen mit lokal variierender Dichte und expliziter Rauschbehandlung (HDBSCAN) zu erfassen (Tsiptsis und Chorianopoulos (2010), S. 85–86; Chakraborty et al. (2022), S. 87–96).

K-Means: Partitionierung mit Zentroiden

K-Means „starts with an initial cluster solution which is updated and adjusted until no further refinement is possible ... Each iteration refines the solution by reducing the within-cluster variation“ (Tsiptsis und Chorianopoulos (2010), S. 85). Die Anzahl der Cluster k wird vorab festgelegt; jedes Cluster wird durch sein Zentrum (Zentroid) repräsentiert, typischerweise unter euklidischer Distanz. Damit liefert K-Means klar interpretierbare **Zentroidprofile**, ist aber am stärksten dort, wo Cluster kompakt und ungefähr kugelförmig sind (Tsiptsis und Chorianopoulos (2010), S. 85–86).

Für die Anfangswahl von k wird zunächst eine kleine Bandbreite getestet (z. B. $k = 3-8$); die **tatsächliche** Clusterzahl wird anschließend über interne Gütemaße (u. a. Silhouette, Davies–Bouldin) sowie Stabilität entschieden (vgl. Abschnitt @sec-evaluation). Ergänzend verweisen neuere Arbeiten auf Varianten und Initialisierungsstrategien, die Robustheit und Konvergenz verbessern (Lai et al. (2025)).

HDBSCAN: Dichtebasierte Hierarchie mit Rauschen

Dichtebasierte Ansätze identifizieren Cluster als Bereiche **hoher Punktdichte** und weisen dünn besetzte Regionen als **Rauschen** aus; klassisch erfolgt dies über `MinPts` (DBSCAN). **HDBSCAN** erweitert dieses Prinzip hierarchisch, extrahiert stabile Cluster über ein Spektrum von Dichteschwellen und **erfordert keine feste Vorgabe von k** . Dadurch lassen sich nicht-konvexe Formen, lokal variierende Dichten und Ausreißer natürlicher behandeln (Chakraborty et al. (2022), S. 87–89, 96).

Die Qualitätsbewertung dichtebasierter Lösungen erfolgt mit **DBC**V (Density-Based Clustering Validation), das die „within- and between-cluster density connectedness“ misst und den Einfluss von Rauschen sachgerecht einbezieht (Moulavi et al. (2014), S. 839f).

Warum zwei Verfahren?

- **Modellannahmen:** K-Means adressiert kompakte Cluster und liefert gut lesbare Zentroidprofile (nützlich für Personas/Profiling). HDBSCAN erkennt **arbiträre Formen** und separiert **Rauschen**, was bei E-Commerce-Daten mit Ausreißern/Saisonalitäten vorteilhaft ist (Chakraborty et al. (2022)).
- **Praxisnutzen:** Der Methoden-Kontrast reduziert das Risiko methodischer Scheinsicherheit und erhöht die Wahrscheinlichkeit, robuste, **geschäftlich interpretierbare** Segmente zu finden.
- **Evaluation:** Je Verfahren passende Indizes: **Silhouette** und **Davies–Bouldin** für K-Means sowie **DBC**V für HDBSCAN; Stabilität via Bootstrapping/ARI, Clustertendenz vorab via **Hopkins** (vgl. @sec-validierung und @sec-evaluation; Lai et al. (2025); Moulavi et al. (2014)).

Einbettung in CRISP-DM

Die Modellierung folgt CRISP-DM: gemeinsame Merkmalsbasis (RFM, Warenkorb, Sortiment etc.), dann parallele Kalibrierung beider Verfahren und methodengerechte Validierung; Ergebnisse werden reproduzierbar dokumentiert und für die **Segment-profile/Personas** aufbereitet (Wirth und Hipp (2000)).

4.3 Clustertendenz und Modelle Hopkins, K-Means, HDBSCAN

Bevor konkrete Clusterverfahren eingesetzt werden, wird mit der Hopkins-Statistik geprüft, ob die Daten überhaupt eine erkennbare Tendenz zur Bildung von Clustern aufweisen. Werte nahe 0,5 deuten auf eine gleichmäßige Streuung ohne klare Gruppen hin, während Werte oberhalb von 0,75 auf deutliche Clustermuster schließen lassen. Vorgehen, Stichprobenumfang, Reproduzierbarkeit und Ergebniszusammenfassung sind in Anhang C.6: Clustertendenz und Testverfahren dokumentiert.

Aufbauend auf dieser Prüfung werden zwei komplementäre Verfahren eingesetzt, die unterschiedliche Strukturannahmen abdecken. K-Means dient als zentroidbasiertes Referenzverfahren und erzeugt gut interpretierbare Zentroidprofile, ist aber auf annähernd kugelförmige Strukturen ausgerichtet. Exemplarisch wird zunächst ein Startwert von $k = 5$ verwendet, um die Funktionsweise zu demonstrieren. Eine inhaltliche Entscheidung über die tatsächliche Clusterzahl erfolgt hier nicht, sondern systematisch in Kapitel 4: Evaluation und Ergebnisse anhand geeigneter Gütemaße (u. a. Silhouette, Davies-Bouldin) und Stabilitätsprüfungen. Die konkreten Umsetzungsschritte stehen in Anhang C.7: Modellierungsschritte K-Means und HDBSCAN.

Als dichte-basiertes Gegenstück wird HDBSCAN eingesetzt. Das Verfahren erfordert keine feste Clusterzahl, kann lokal unterschiedliche Dichten abbilden und weist dünn besetzte Bereiche als Rauschen aus. Dadurch werden neben größeren Segmenten auch Nischenmuster identifiziert. Parameterwahl und Export der Ergebnisse sind ebenfalls in Anhang C.7 dokumentiert; die Qualität der resultierenden Lösungen wird in Kapitel 4 anhand dichtegeeigneter Indizes und Stabilitätsmaßen bewertet.

Damit ist die methodische Basis gelegt. Die Hopkins-Statistik begründet die Durch-

führung einer Clusteranalyse, und die Kombination aus K-Means und HDBSCAN stellt sicher, dass sowohl zentroidbasierte als auch dichtebasierte Strukturen erfasst werden.

4.4 Validierung (Hopkins, Silhouette, DBCV) als Qualitätskontrolle

Damit die mit Clustering identifizierten Muster belastbar sind, müssen ihre Ergebnisse überprüft werden. Dazu dienen Validierungsmaße, die im Folgenden vorgestellt werden.

Die Hopkins-Statistik prüft, ob ein Datensatz überhaupt eine Clustertendenz enthält. Acito (2023), S. 271 beschreiben sie als Maß, „to assess the clustering tendency of a data set by measuring the probability that a uniform data distribution generates a given data set“. Werte über 0,75 deuten auf eine starke Clustertendenz hin, Werte nahe 0,5 auf Zufälligkeit. Bei hochdimensionalen E-Commerce-Daten ist die Aussagekraft jedoch eingeschränkt, weshalb Hopkins in dieser Arbeit nur ergänzend berücksichtigt wird.

Der Silhouette-Koeffizient bewertet die Kohäsion innerhalb von Clustern und die Separation zwischen Clustern. Er kombiniert also zwei Perspektiven: interne Homogenität und externe Abgrenzung. Tsiptsis und Chorianopoulos (2010), S. 209 nennen die Silhouette als ein zentrales Kriterium der technischen Evaluation. Lai et al. (2025), S. 3063 ergänzen, dass sie ein objektives Maß sei, weil sie „both the cohesion within clusters and the separation between clusters“ berücksichtigt.

Für dichtebasierte Verfahren wie HDBSCAN ist der DBCV-Index geeignet. Moulavi et al. (2014), S. 839 erklären, dass DBCV die „within and between-cluster density connectedness“ misst, während Rauschen explizit berücksichtigt wird. Der Vorteil liegt darin, dass dieses Maß auch für arbiträre Clusterformen robust anwendbar ist (Moulavi et al. (2014), S. 845).

Dabei ist zu berücksichtigen, dass interne Validierungsmaße wie Silhouette oder DBCV lediglich strukturelle Eigenschaften der Clusterlösungen abbilden. Sie garantieren nicht, dass die Segmente geschäftlich relevant sind. Ohne eine anschließende

inhaltliche Interpretation besteht die Gefahr, rechnerisch erzeugte Gruppen zu überinterpretieren.

Diese theoretischen Grundlagen bilden den Rahmen, um in der folgenden Untersuchung herauszuarbeiten, welche Segmentstrukturen sich in E-Commerce-Daten verbergen und wie sie so beschrieben werden können, dass sie für Marketing und CRM unmittelbar nutzbar sind.

5 Ergebnisse und Diskussion (CRISP-DM: Modellierung und Evaluation - praktisch)

5.1 Preprocessing (Bereinigung, Skalierung, Variablenableitung – Resultate, Details in Anhang C)

Die Datensätze wurden zunächst bereinigt, Duplikate und fehlerhafte Einträge entfernt sowie Rückgaben separat behandelt. Anschließend erfolgte die Vereinheitlichung der Zeitstempel und die Konstruktion abgeleiteter Variablen ($\text{Umsatz} = \text{Preis} \times \text{Menge}$). Über Recency, Frequency und Monetary Value hinaus wurden Merkmale wie Warenkorbgröße und Sortimentsvielfalt gebildet. Zur Vergleichbarkeit aller Variablen wurde eine Standardisierung angewandt. Die detaillierte Umsetzung und die Explorative Datenanalyse sind in Anhang C dokumentiert.

5.2 Clustering (K-Means und HDBSCAN – Ergebnisse und Validierung)

Zur Segmentierung wurden zwei Verfahren angewandt:

K-Means: Für verschiedene Clusterzahlen ($k = 3-8$) berechnet. Die Silhouette-Werte zeigten, dass eine 5-Cluster-Lösung eine gute Balance aus Trennschärfe und Interpretierbarkeit liefert.

HDBSCAN: Automatische Erkennung von Clustern mit variabler Dichte. Neben drei stabilen Segmenten wurde ein Teil der Kunden als Rauschen identifiziert. Der DBCV-Wert bestätigte eine robuste Abgrenzung der Cluster.

Die Validierung verdeutlicht, dass K-Means für kompakte Strukturen geeignet ist, während HDBSCAN flexibler auf unregelmäßige Muster reagiert.

5.3 Interpretation der Segmente (Clusterprofile, Personas, Visualisierungen)

Die Cluster wurden anhand zentraler Kennzahlen interpretiert:

Cluster 1: Häufige Käufer mit mittlerem Warenkorbwert (loyale Stammkunden).

Cluster 2: Seltene Käufer mit hohen Umsätzen (Gelegenheitskunden mit Premium-Fokus).

Cluster 3: Wenige Käufe, geringer Umsatz (Kunden mit Abwanderungsrisiko).

Cluster 4: Sehr hohe Frequenz, kleine Warenkörbe (Schnäppchenjäger).

Cluster 5 (HDBSCAN): Unregelmäßiges Kaufmuster, oft saisonal bedingt (Gelegenheitsnutzer).

Diese Segmente wurden in Personas übersetzt und durch Visualisierungen unterstützt (siehe Anhang D). Daraus lassen sich für Marketing und CRM differenzierte Maßnahmen ableiten, z. B. Reaktivierungskampagnen für Cluster 3 oder exklusive Angebote für Cluster 2.

6 Diskussion

6.1 Vergleich der Verfahren

6.2 Implikationen für Marketing und CRM

6.3 Grenzen und methodische Limitationen

7 Fazit und Ausblick

7.1 Beantwortung der Forschungsfrage

Die Arbeit hat gezeigt, dass sich in E-Commerce-Daten mehrere klar unterscheidbare Segmente identifizieren lassen. Diese unterscheiden sich sowohl in quantitativen Kennzahlen wie Kaufhäufigkeit, Umsatzhöhe und Sortimentspräferenzen als auch in charakteristischen Verhaltensmustern. Mit den aufbereiteten Profilen, Personas und Visualisierungen konnten die Unterschiede verständlich gemacht und in praxisnahe Maßnahmen überführt werden. Damit wurde die Forschungsfrage beantwortet, wie Segmentstrukturen in E-Commerce-Daten sichtbar gemacht und für Marketing und CRM nutzbar aufbereitet werden können.

Die eingesetzten Verfahren haben dabei ihren Zweck erfüllt. K-Means und HDBSCAN dienten als Werkzeuge, um Strukturen sichtbar zu machen. Silhouette und DBCV überprüften die Qualität der Ergebnisse. Die Hopkins-Statistik wurde ergänzend einbezogen, zeigte erwartungsgemäß kaum Clusterfähigkeit und machte deutlich, dass manche Validierungsansätze in diesem Kontext methodisch nur begrenzt aussagekräftig sind.

7.2 Limitationen

Die Ergebnisse basieren auf internen Validitätsmaßen und bieten keinen direkten Nachweis, ob segmentbasierte Maßnahmen in der Praxis zu besseren Erfolgen führen. Auch die Datenbasis weist Einschränkungen auf. Transaktions- und Eventdaten sind häufig unbalanciert, enthalten Ausreißer und bilden nicht alle relevanten Kundenmerkmale

ab. Die Generalisierbarkeit auf andere Märkte oder Shops ist daher begrenzt.

7.3 Ausblick (z. B. Text Mining, Echtzeit-Segmentierung)

Für zukünftige Arbeiten empfiehlt sich die Ergänzung um externe Validierung. A/B-Tests und Uplift-Analysen könnten zeigen, ob segmentierte Kampagnen tatsächlich bessere Ergebnisse liefern. Eine Erweiterung der Merkmalsbasis durch Text Mining von Kundenbewertungen oder die Analyse von Session-Daten könnte zusätzliche Differenzierung schaffen. Auch eine dynamische Segmentierung, die sich in Echtzeit an Veränderungen im Verhalten anpasst, wäre ein sinnvoller Schritt. Schließlich sollte die Segmentlogik direkt in CRM-Systeme integriert werden, um einen geschlossenen Kreislauf zwischen Analyse, Umsetzung und Erfolgsmessung zu schaffen.

Was eine [Multi-Faktor-Authentifizierung \(MFA\)](#) ist, wird im Glossar beschrieben.

Literaturverzeichnis

- Acito, F. (2023). *Predictive Analytics with KNIME: Analytics for Citizen Data Scientists*. Springer Nature Switzerland. <https://doi.org/10.1007/978-3-031-45630-5>
- Chakraborty, S., Islam, S. H., und Samanta, D. (2022). *Data Classification and Incremental Clustering in Data Mining and Machine Learning*. Springer International Publishing. <https://doi.org/10.1007/978-3-030-93088-2>
- Datasets - UCI Machine Learning Repository*. (2019, September 21). https://archive.ics.uci.edu/datasets?search=Online+Retail&utm_source=chatgpt.com
- Lai, H., Huang, T., Lu, B., Zhang, S., und Xiaog, R. (2025). Silhouette coefficient-based weighting k-means algorithm. *Neural Computing and Applications*, 37(5), 3061–3075. <https://doi.org/10.1007/s00521-024-10706-0>
- Moulavi, D., Jaskowiak, P. A., Campello, R. J. G. B., Zimek, A., und Sander, J. (2014). Density-Based Clustering Validation. *Proceedings of the 2014 SIAM International Conference on Data Mining*, 839–847. <https://doi.org/10.1137/1.9781611973440.96>
- Tsiptsis, K., und Chorianopoulos, A. (2010). *Data Mining Techniques in CRM: Inside Customer Segmentation* (1. Aufl.). Wiley. <https://doi.org/10.1002/9780470685815>
- Wirth, R., und Hipp, J. (2000). *CRISP-DM: Towards a Standard Process Model for Data Mining*. 11. <http://cs.unibo.it/~danilo.montesi/CBD/Beatriz/10.1.1.198.5133.pdf>
- Zykov, R., Noskov, A., und Anokhin, A. (2022). *Retailrocket recommender system dataset*. Kaggle.com. <https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset>

Anhang

Hinweis: Alle Abbildungen in den Anhängen tragen das Präfix **A** in ihrer Nummerierung. Das **A** verweist zugleich auf den Anhang und wird fortlaufend gezählt.

Anhangsverzeichnis

Anhang A Modelle und Prozesse	26
A.1 Das CRISP-DM-Modell	27
A.2 Segmentation Management Strategy	28
Anhang B Clustering und RFM	30
B.1 Eindimensionale RFM-Segmentierung	32
B.2 Mehrdimensionale RFM-Segmentierung	32
B.3 Retail-Segmentierung mit RFM	33
Anhang C Data Preparation	51
C.1 Bereinigung (fehlende Werte, Duplikate, Rückgaben)	52
C.2 Skalierung und Transformation	56
C.3 Explorative Datenanalyse (zusätzliche Plots, Tabellen, Code)	60

Anhang A: Prozessmodelle

Dieser Anhang stellt zwei etablierte Prozessmodelle vor, die für Datenanalyse und Segmentierungsstrategien maßgeblich sind. Mit dem CRISP-DM-Referenzmodell wird ein methodischer Rahmen für die systematische Durchführung von Data-Mining-Projekten beschrieben. Ergänzend veranschaulicht die Segmentation Management Strategy, wie analytisch gewonnene Kundensegmente in strategische Marketing- und CRM-Maßnahmen überführt werden können. Die Kombination macht deutlich, dass erfolgreiche Segmentierung sowohl methodisch fundiert als auch organisatorisch eingebettet sein muss.

A.1: Das CRISP-DM-Modell

Die Abbildung A.1 zeigt das CRISP-DM-Referenzmodell mit seinen sechs Phasen Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation und Deployment. Pfeile markieren die zentralen Abhängigkeiten, während der äußere Kreis den iterativen, zyklischen Charakter des Vorgehens unterstreicht. Das Modell ist als allgemeiner Rahmen entworfen und unabhängig von Anwendungsdomäne oder eingesetzter Software.

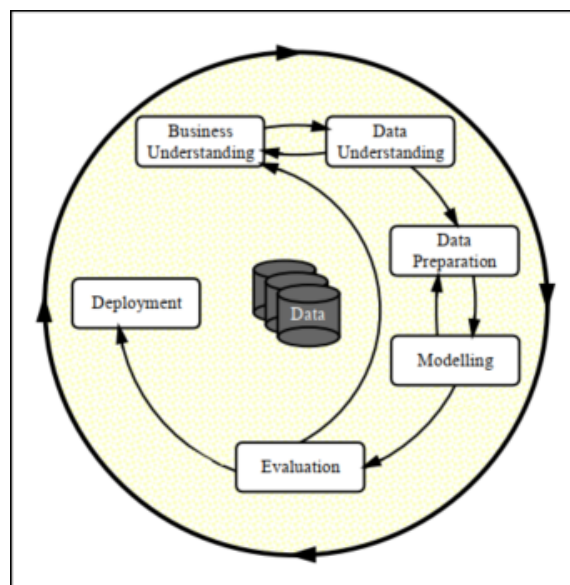


Abbildung A.1: Das CRISP-DM-Modell nach Wirth und Hipp (2000), S. 5

Jede der sechs Phasen ist in spezifische Aufgaben untergliedert, die den Ablauf systematisieren und nachvollziehbar machen. So umfasst Data Understanding das Beschreiben, Erkunden und Überprüfen der Datenqualität, während Data Preparation Aufgaben wie Auswählen, Bereinigen, Konstruieren und Integrieren von Daten vorsieht. Modeling beinhaltet die Wahl geeigneter Verfahren, die Erstellung eines Testdesigns sowie den Aufbau und die Bewertung von Modellen. Evaluation konzentriert sich auf die Überprüfung der Ergebnisse im Hinblick auf die Projektziele, und Deployment behandelt die Bereitstellung und Nutzung der Modelle im Anwendungskontext. Wirth und Hipp betonen außerdem den stark iterativen Charakter des Modells: Ergebnisse

einer Phase können jederzeit zu Rücksprüngen führen, wodurch die Arbeitsschritte flexibel angepasst werden können (Wirth und Hipp (2000), S. 6–9).

A.2: Segmentation Management Strategy

Die in Abbildung A.2 dargestellte Roadmap der *Segmentation Management Strategy* zeigt, wie analytisch ermittelte Segmente in eine unternehmerische Nutzung überführt werden können. Sie ergänzt das CRISP-DM-Modell, indem sie die Brücke von der technischen Modellierung hin zur strategischen Implementierung schlägt. Während CRISP-DM beschreibt, wie Daten zu Modellen verarbeitet werden, zeigt die Segmentation Management Strategy, wie diese Modelle in Marketing- und CRM-Maßnahmen eingebettet werden.

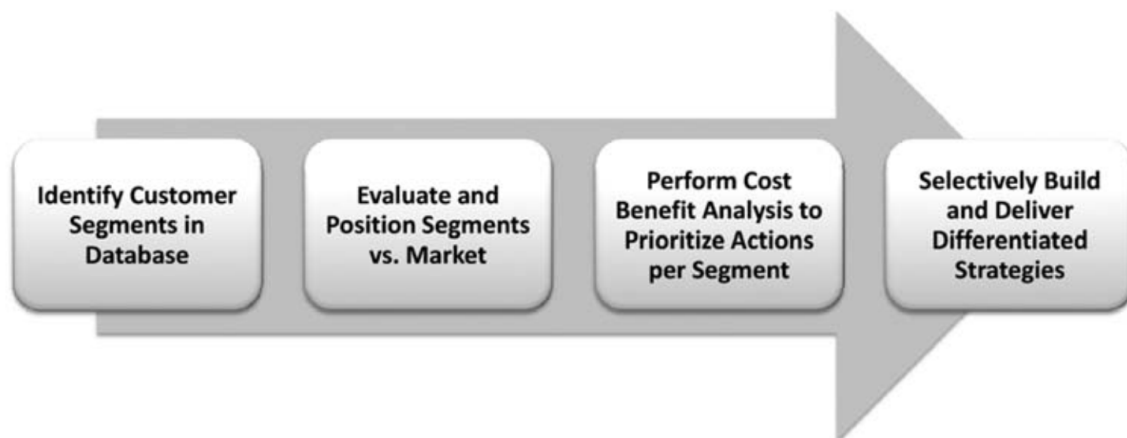


Abbildung A.2: Roadmap der Segmentation Management Strategy nach Tsipsis und Chorianopoulos (2010), S. 213–216

Die Roadmap besteht aus vier Schritten, die nacheinander zu durchlaufen sind:

Schritt 1: Identify the customer segments in the database

Im ersten Schritt werden die ermittelten Segmente in die Kundendatenbank integriert. Kunden erhalten Scores und werden den Segmenten zugewiesen. Tsipsis und Chorianopoulos (2010), S. 213 betonen: *“Initially the segmentation results should be deployed in the customer database. Customers should be scored and assigned into segments.”* Damit ist die organisatorische Grundlage geschaffen, auf der weitere Analysen und

Maßnahmen aufbauen.

Schritt 2: Evaluate and position the segments

In einem zweiten Schritt werden die Segmente bewertet und im Marktumfeld positioniert. Dazu gehören Marktumfragen und qualitative Analysen, die Segmente mit zusätzlichen Informationen wie Bedürfnissen, Lebensstil oder Status anreichern. Zudem sollen Wettbewerbsvorteile, Chancen und Risiken identifiziert werden. Die Autoren halten fest, dass *“the possible opportunities and threats should be investigated in order to select the key segments to be targeted with marketing activities tailored to their profiles and needs”* (Tsiptsis und Chorianopoulos (2010), S. 214).

Schritt 3: Perform cost-benefit analysis to prioritize actions per segment

Darauf folgt eine Kosten-Nutzen-Analyse, die die Profitabilität einzelner Segmente in Relation zu den erwarteten Kosten setzt. Behavioral Drivers wie Kaufhäufigkeit oder Abwanderungsrisiken spielen dabei eine zentrale Rolle. Ziel ist die Priorisierung der Segmente nach ökonomischer Relevanz. *“Effective segmentation management requires insight into the factors that drive customer behaviors ... Finally, a cost-benefit analysis can enable the prioritization of the marketing actions to be implemented”* (Tsiptsis und Chorianopoulos (2010), S. 215).

Schritt 4: Build and deliver differentiated strategies

Im vierten Schritt werden die Segmente mit maßgeschneiderten Strategien adressiert. Spezialisierte Teams entwickeln und implementieren differenzierte Kampagnen, CRM-Maßnahmen und Produktstrategien. Dies umfasst Aspekte wie Kanalmanagement, Marketingservices, Produktmanagement und Markenführung. Tsiptsis und Chorianopoulos (2010), S. 216 schreiben hierzu: *“Each team should build and deliver specialized marketing strategies to improve customer handling, develop the relationship with customers, and increase the profitability of each segment.”*

Die Roadmap verdeutlicht, dass Segmentierung mehr ist als ein analytischer Prozessschritt. Sie muss in eine organisatorische und strategische Logik eingebettet werden, damit Segmente nicht nur statistisch existieren, sondern im Unternehmen handlungs-

leitend wirken. CRISP-DM liefert dafür den methodischen Rahmen, die Segmentation Management Strategy ergänzt ihn um eine explizite Managementperspektive.

Anhang B: Clustering und RFM

Dieser Anhang erläutert die RFM-Segmentierung als ein klassisches Instrument der Kundenanalyse. Im Mittelpunkt stehen die Dimensionen Recency, Frequency und Monetary Value, mit denen Kundenverhalten in strukturierte Kategorien überführt werden kann. Die Darstellung zeigt sowohl einfache eindimensionale Ansätze als auch mehrdimensionale Ausprägungen, die eine differenziertere Sicht auf Kaufmuster und Kundenwert ermöglichen. Damit wird nachvollziehbar, weshalb RFM-Modelle eine wichtige Grundlage für moderne Verfahren der Kundensegmentierung bilden.

B.1: Eindimensionale RFM-Segmentierung

Eindimensionale RFM-Segmentierung betrachtet jeweils nur eine Dimension des RFM-Modells – also Recency, Frequency oder Monetary – isoliert.

Dabei werden Kunden typischerweise in Quantile eingeteilt. So werden beispielsweise die 20 % der Kunden mit der höchsten Kaufhäufigkeit in die oberste Kategorie ($F = 5$) eingeordnet, während die 20 % mit der geringsten Häufigkeit die niedrigste Stufe ($F = 1$) bilden (Tsiptsis und Chorianopoulos, 2010, S. 333–336).

Dieser Ansatz ist leicht verständlich und ermöglicht eine schnelle Einordnung von Kunden nach einem einzelnen Aspekt des Kaufverhaltens. Allerdings bildet er die Gesamtkomplexität der Kundenbeziehungen nur eingeschränkt ab, da Interaktionen zwischen den Dimensionen unberücksichtigt bleiben.

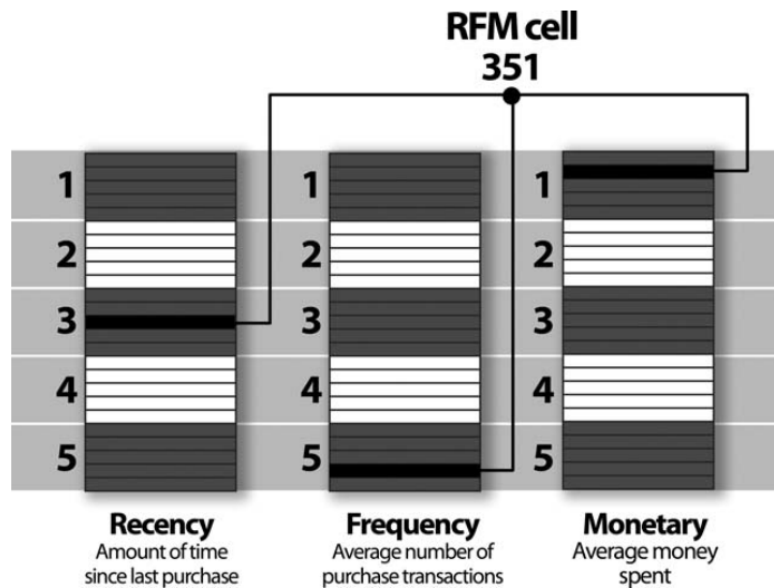


Abbildung A.3: Beispiel für eindimensionale RFM-Einteilungen in fünf Quantile je Dimension. Hervorgehoben ist die RFM-Zelle 351 (Recency = 3, Frequency = 5, Monetary = 1) nach Tsiptsis und Chorianopoulos (2010), S. 336.

B.2: Mehrdimensionale RFM-Segmentierung

Die RFM-Analyse kombiniert drei Dimensionen: Recency, Frequency und Monetary Value. Tsiptsis und Chorianopoulos (2010), S. 336–345 beschreiben RFM als bewährten Ansatz, um Kundenwerte differenziert abzubilden. Neben der klassischen Zellenbildung (z. B. Quintile) kann RFM auch als Input für Clustering genutzt werden. Dadurch entstehen mehrdimensionale Segmente, die sich deutlich voneinander unterscheiden. Erweiterungen des Modells beziehen zusätzliche Merkmale wie Warenkorbgrößen oder Sortimentspräferenzen ein, um die Aussagekraft weiter zu erhöhen.

B.3: Retail-Segmentierung mit RFM

Ergänzend zur theoretischen Einführung im Hauptteil wird hier die Zellenlogik von RFM veranschaulicht.

Die folgende programmgenerierte Darstellung zeigt alle möglichen Kombinationen von *Recency* und *Frequency* auf einer Ebene mit festem *Monetary*-Wert. Die Zelle 351 ist dabei als Beispiel hervorgehoben.

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(12, 9))
ax = fig.add_subplot(111, projection='3d')

# RFM-Dimensionen
recency_range = range(1, 6)      # 1-5
frequency_range = range(1, 6)    # 1-5
monetary_level = 1                # Feste Ebene bei Z=1

# Vollständige Ebene bei Monetary = 1
for r in recency_range:
    for f in frequency_range:
        code = f"{r}-{f}-{monetary_level}"
```

```

# Hervorhebung für Code "351"
if code == "351":
    color = "red"          # Besondere Hervorhebung in rot
    text_color = "white"   # Weiße Schrift auf rot
else:
    color = "lightgrey"
    text_color = "black"   # Schwarze Schrift auf grau

# Korrigierte Positionierung: x=r, y=f, z=monetary_level
ax.bar3d(
    x=r-0.5, y=f-0.5, z=monetary_level-0.1, # Startposition (Ecke des
Würfels)
    dx=1, dy=1, dz=0.2,                    # Größe
    color=color, edgecolor='black', alpha=0.9 # Weniger transparent
)

ax.text(
    r, f, monetary_level + 0.05, # Leicht über der Oberfläche
    code,
    color=text_color,
    ha='center', va='center',
    fontsize=14,                    # Noch größere Schrift
    weight='bold',                 # ALLE Texte fett
    zorder=100                     # Text immer im Vordergrund
)

# Achsentitel (korrekte Zuordnung)
ax.set_xlabel("Recency", fontsize=12, weight='bold')
ax.set_ylabel("Frequency", fontsize=12, weight='bold')
ax.set_zlabel("Monetary", fontsize=12, weight='bold')

# Achsenticks
ax.set_xticks(range(1, 6))

```

```
ax.set_xticklabels(['1', '2', '3', '4', '5'])
ax.set_yticks(range(1, 6))
ax.set_yticklabels(['1', '2', '3', '4', '5'])
ax.set_zticks(range(1, 6))

# Achsgrenzen
ax.set_xlim(0.5, 5.5)
```

```
ax.set_ylim(0.5, 5.5)
```

```
ax.set_zlim(0.5, 5.5)
```

```
# Seitenverhältnis & Perspektive
ax.set_box_aspect([1, 1, 1]) # Gleiche Proportionen
ax.view_init(elev=25, azim=45) # Bessere Sicht
ax.grid(True)

# Überschrift mit RFM-Score Berechnung
ax.set_title("RFM-Würfel: Kontinuierlicher Score-Ansatz\n" +
             "RFM Score = (R×wR) + (F×wF) + (M×wM)\n" +
             "Ebene bei Monetary = 1 | Code 351 hervorgehoben",
             fontsize=13, weight='bold', pad=20)

# Layout-Anpassung ohne Warnung
plt.subplots_adjust(left=0.1, right=0.9, top=0.85, bottom=0.1) # Mehr Platz
für Titel
plt.show()
```

RFM-Würfel: Kontinuierlicher Score-Ansatz
 $\text{RFM Score} = (R \times w_R) + (F \times w_F) + (M \times w_M)$
 Ebene bei Monetary = 1 | Code 351 hervorgehoben

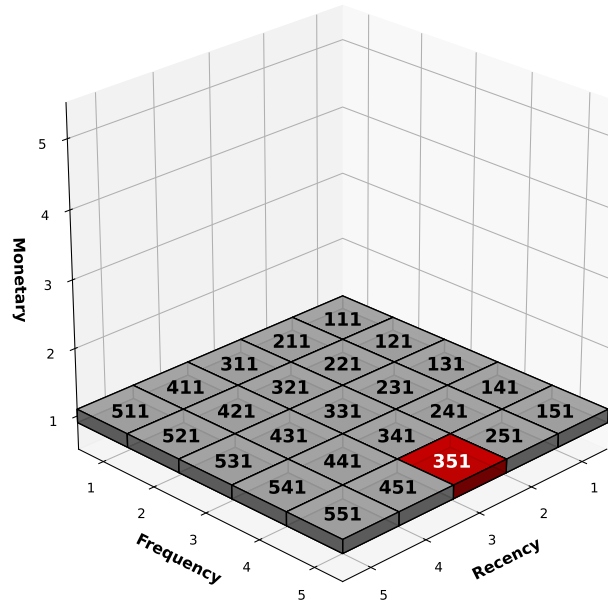


Abbildung A.4: Eigene Darstellung eines RFM-Würfels

Die Abbildung zeigt einen zweidimensionalen Ausschnitt des RFM-Modells in Würfelform:

- Die x-Achse steht für *Recency* (Zeit seit dem letzten Kauf).
- Die y-Achse repräsentiert *Frequency* (Kaufhäufigkeit).
- Die z-Achse bildet *Monetary* (Umsatzhöhe) ab. In dieser Darstellung ist die Ebene mit *Monetary* = 1 ausgewählt.

Jede Zelle im Würfel steht für eine Kombination dieser drei Dimensionen und wird durch einen dreistelligen Code gekennzeichnet (z. B. 351 = Recency 3, Frequency 5, Monetary 1).

Die hervorgehobene Zelle 351 veranschaulicht exemplarisch, wie einzelne Kundengruppen identifiziert werden können. Damit wird deutlich, dass das RFM-Schema ein Raster bildet, das eine differenzierte Beschreibung von Kundenverhalten ermöglicht.

B.4: Warenkorb- und Sortimentsmerkmale

Dieser Abschnitt illustriert die im Hauptteil beschriebenen Konzepte *Warenkorbgrößen* und *Sortimentsmerkmale* anhand von Beispielen und Visualisierungen. Während Kapitel 4.1 die theoretischen Grundlagen und formalen Definitionen eingeführt hat, werden hier exemplarische Kundentypen wie „Frequent Shopper“, „Bulk Buyer“, „Generalisten“ und „Spezialisten“ dargestellt. Auf diese Weise wird deutlich, wie sich die abstrakten Kennzahlen in der Praxis auf konkrete Verhaltensmuster übertragen lassen.

Warenkorbgrößen

Die Abbildung A.5 zeigt zwei kontrastierende Kundentypen:

- Kunde A („Frequent Shopper“) tätigt häufige, kleine Einkäufe.
- Kunde B („Bulk Buyer“) kauft selten, aber in großen Mengen.

Obwohl beide die gleiche Gesamtzahl an Artikeln kaufen, unterscheiden sie sich im Bestellverhalten fundamental. Diese Unterschiede lassen sich durch die *Average Basket Size* quantifizieren.

```
import matplotlib.pyplot as plt
import matplotlib.path_effects as path_effects
import numpy as np

# Daten für beide Kundentypen
customers = ["Kunde A\n(Frequent Shopper)", "Kunde B\n(Bulk Buyer)"]
basket_size = [2, 20]
order_count = [10, 2]
total_items = [20, 40]

# 3D-Farbpalette
colors_primary = ['#FF6B6B', '#4ECDC4']
colors_secondary = ['#E74C3C', '#45B7D1']
```

```

colors_scatter = ['#FF1744', '#00BCD4']

fig = plt.figure(figsize=(16, 10))

# Layout mit GridSpec - ERWEITERT für Kennzahlen
gs = fig.add_gridspec(3, 3, height_ratios=[2.5, 1, 0.8], width_ratios=[1, 1,
0.8],
                    hspace=0.4, wspace=0.3, top=0.88, bottom=0.08)

# 3D Bar Charts
ax1 = fig.add_subplot(gs[0, 0])
ax2 = fig.add_subplot(gs[0, 1])
ax3 = fig.add_subplot(gs[1, 0])
ax4 = fig.add_subplot(gs[:2, 2]) # Rechts über 2 Zeilen
ax5 = fig.add_subplot(gs[2, :2]) # NEUE Kennzahlen-Zeile

# 1. Warenkorbgröße mit besserer Y-Achsen-Skalierung
bars1 = ax1.bar(customers, basket_size, color=colors_primary, alpha=0.9,
                edgecolor='black', linewidth=2,
                width=0.6)
ax1.set_title('Durchschnittliche Warenkorbgröße', fontweight='bold',
             fontsize=12, pad=15)
ax1.set_ylabel('Artikel pro Bestellung', fontweight='bold')
ax1.grid(True, alpha=0.3, linestyle='--')
ax1.set_facecolor('#F8F9FA')
ax1.set_ylim(0, max(basket_size) * 1.3)

```

Terminal-Output

(0.0, 26.0)

```

# Werte besser positioniert
for i, (bar, value) in enumerate(zip(bars1, basket_size)):
    text = ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
max(basket_size) * 0.05,

```

```

        str(value), ha='center', va='bottom', fontweight='bold',
fontsize=14,
        color='white')
    text.set_path_effects([path_effects.withStroke(linewidth=2,
foreground='black')])

# 2. Anzahl Bestellungen mit besserer Y-Achsen-Skalierung
bars2 = ax2.bar(customers, order_count, color=colors_secondary, alpha=0.9,
                edgecolor='black', linewidth=2,
                width=0.6)
ax2.set_title('Anzahl Bestellungen', fontweight='bold', fontsize=12, pad=15)
ax2.set_ylabel('Bestellungen gesamt', fontweight='bold')
ax2.grid(True, alpha=0.3, linestyle='--')
ax2.set_facecolor('#F8F9FA')
ax2.set_ylim(0, max(order_count) * 1.3)

```

Terminal-Output

(0.0, 13.0)

```

# Werte besser positioniert
for i, (bar, value) in enumerate(zip(bars2, order_count)):
    text = ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
max(order_count) * 0.05,
                    str(value), ha='center', va='bottom', fontweight='bold',
fontsize=14,
                    color='white')
    text.set_path_effects([path_effects.withStroke(linewidth=2,
foreground='black')])

# 3. Scatter Plot GRÖßER mit besserer Achsen-Skalierung
ax3.scatter(order_count, basket_size, s=[500, 500],
            c=colors_scatter, alpha=0.8,
            edgecolors='black', linewidth=2,

```

```

        zorder=5)
ax3.set_xlabel('Anzahl Bestellungen', fontweight='bold', fontsize=11)
ax3.set_ylabel('Warenkorbgröße', fontweight='bold', fontsize=11)
ax3.set_title('Kundentypen im Vergleich', fontweight='bold', fontsize=13,
pad=15)
ax3.grid(True, alpha=0.3, linestyle='--')
ax3.set_facecolor('#F8F9FA')

# Bessere Achsen-Grenzen für mehr Platz
ax3.set_xlim(0, max(order_count) * 1.2)

```

Terminal-Output

(0.0, 12.0)

```
ax3.set_ylim(0, max(basket_size) * 1.2)
```

Terminal-Output

(0.0, 24.0)

```

# Annotationen mit 3D-Effekt - größer
for i, customer in enumerate(['A', 'B']):
    ax3.annotate(f'Kunde {customer}',
                xy=(order_count[i], basket_size[i]),
                xytext=(10, 10), textcoords='offset points',
                fontweight='bold', fontsize=12, color='white',
                bbox=dict(boxstyle="round,pad=0.4",
                        facecolor=colors_scatter[i], alpha=0.8,
                        edgecolor='black', linewidth=2))

# 4. Strategische Implikationen mit kompakterem Layout
ax4.axis('off')

```


Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```
# 3D-artiger Hintergrund
ax4.add_patch(plt.Rectangle((0, 0), 1, 1, transform=ax4.transAxes,
                           facecolor='lightgray', alpha=0.2,
                           edgecolor='black', linewidth=1))

# Titel mit 3D-Box
ax4.text(0.05, 0.95, 'Strategische Implikationen', fontsize=12,
        fontweight='bold',
        transform=ax4.transAxes, color='white',
        bbox=dict(boxstyle="round,pad=0.3", facecolor='steelblue', alpha=0.8,
        edgecolor='black'))

# Kunde A Implikationen - kompakter
ax4.text(0.05, 0.85, 'Kunde A (Frequent Shopper):', fontsize=10,
        fontweight='bold',
        transform=ax4.transAxes, color='white',
        bbox=dict(boxstyle="round,pad=0.2", facecolor='#FF6B6B', alpha=0.8,
        edgecolor='black'))

implications_a = ['• Häufige Kontaktpunkte', '• Loyalitätsprogramme', '•
Cross-Selling-Potenzial']
for i, impl in enumerate(implications_a):
    ax4.text(0.05, 0.79 - i*0.05, impl, fontsize=9, transform=ax4.transAxes,
            bbox=dict(boxstyle="round,pad=0.15", facecolor='white', alpha=0.8,
            edgecolor='gray'))

# Kunde B Implikationen - kompakter
ax4.text(0.05, 0.60, 'Kunde B (Bulk Buyer):', fontsize=10, fontweight='bold',
        transform=ax4.transAxes, color='white',
        bbox=dict(boxstyle="round,pad=0.2", facecolor='#4ECDC4', alpha=0.8,
        edgecolor='black'))
```

```

implications_b = ['• Planungsorientiert', '• Volumenrabatte', '•
Lagerservice']
for i, impl in enumerate(implications_b):
    ax4.text(0.05, 0.54 - i*0.05, impl, fontsize=9, transform=ax4.transAxes,
             bbox=dict(boxstyle="round,pad=0.15", facecolor='white', alpha=0.8,
edgecolor='gray'))

# Fazit mit 3D-Effekt - kompakter
ax4.text(0.05, 0.35, 'Fazit:', fontsize=10, fontweight='bold',
         transform=ax4.transAxes, color='white',
         bbox=dict(boxstyle="round,pad=0.2", facecolor='darkgreen', alpha=0.8,
edgecolor='black'))

ax4.text(0.05, 0.25, 'Beide Typen wertvoll, aber\ndunterschiedliche Ansprache
nötig',
         fontsize=9, transform=ax4.transAxes, style='italic',
         bbox=dict(boxstyle="round,pad=0.15", facecolor='lightyellow',
alpha=0.8, edgecolor='orange'))

# NEUE Kennzahlen-Ausgabe für Warenkorbanalyse
ax5.axis('off')

```

Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```

kennzahlen_text = f"""WARENKORBANALYSE: Kunde A - Warenkorbgröße:
{basket_size[0]} Artikel | Bestellungen: {order_count[0]}/Periode | Kunde B -
Warenkorbgröße: {basket_size[1]} Artikel | Bestellungen:
{order_count[1]}/Periode | Verhältnis:
{basket_size[1]/basket_size[0]:.1f}:1"""

ax5.text(0.5, 0.5, kennzahlen_text, fontsize=10,

```

```

transform=ax5.transAxes, ha='center', va='center',
fontfamily='monospace', fontweight='bold',
bbox=dict(boxstyle="round,pad=0.3", facecolor='gold',
           alpha=0.8, edgecolor='black', linewidth=2))

# 3D-Titel höher positioniert
plt.suptitle('Warenkorbstrategien: 3D-Analyse verschiedener Kundentypen',
             fontsize=16, fontweight='bold', y=0.96,
             bbox=dict(boxstyle="round,pad=0.5", facecolor='gold', alpha=0.8,
                       edgecolor='black', linewidth=2))

plt.tight_layout()
plt.show()

```

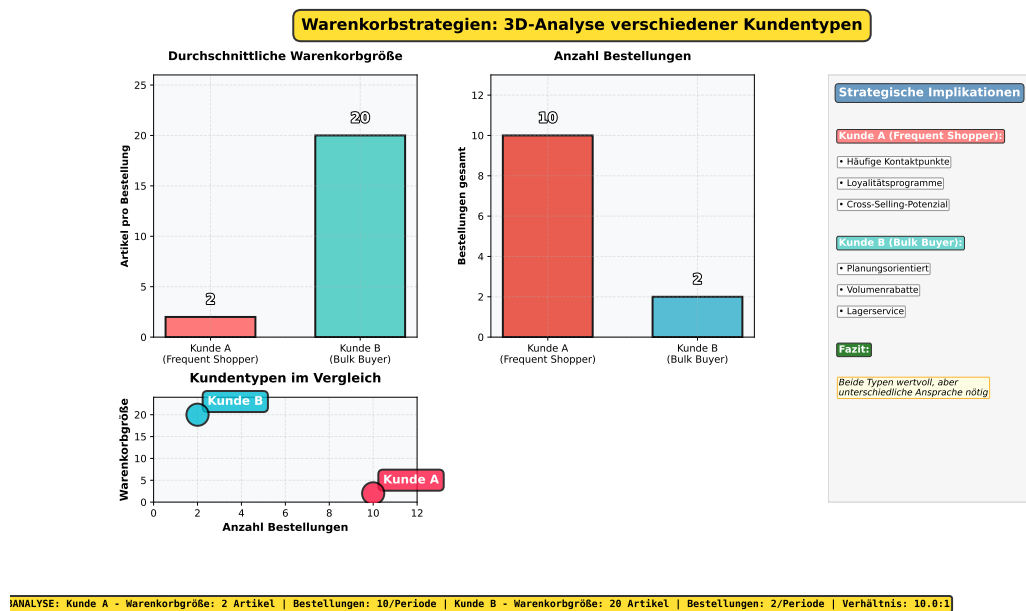


Abbildung A.5: Warenkorbanalyse verschiedener Kundentypen mit Durchschnittswerten, Bestellhäufigkeit und Vergleichsdarstellung

Die Visualisierung verdeutlicht, dass beide Kundentypen für ein Unternehmen wertvoll sein können, jedoch unterschiedliche Marketingstrategien erfordern. Während

Frequent Shopper auf Loyalitätsprogramme und Cross-Selling reagieren, benötigen Bulk Buyer volumenorientierte Anreize und Serviceleistungen.

Sortimentsmerkmale

Die Abbildung A.6 stellt das Verhalten eines Generalisten (Kunde C) und eines Spezialisten (Kunde D) gegenüber.

```
import matplotlib.pyplot as plt
import matplotlib.path_effects as path_effects
import numpy as np

# Beispiel: Sortimentsanteile
categories = ["Elektronik", "Kleidung", "Bücher", "Sport", "Haushalt"]

# Generalist: gleichmäßige Verteilung
generalist = [20, 20, 20, 20, 20]

# Spezialist: starke Fokussierung auf eine Kategorie
specialist = [80, 8, 5, 4, 3]

# Intensivere 3D-Farbpalette mit Verläufen
colors_generalist = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A', '#98D8C8']
colors_specialist = ['#E74C3C', '#3498DB', '#F39C12', '#9B59B6', '#2ECC71']

fig = plt.figure(figsize=(16, 9))

# Layout mit GridSpec - MEHR PLATZ OBEN für Titel
gs = fig.add_gridspec(3, 3, height_ratios=[2.5, 1, 0.8], width_ratios=[1, 1, 0.8],
                      hspace=0.5, wspace=0.3, top=0.82, bottom=0.08) # top reduziert für mehr Platz

# Charts definieren - ALLE WEITER UNTEN
ax1 = fig.add_subplot(gs[0, 0]) # Pie Chart 1
```

```

ax2 = fig.add_subplot(gs[0, 1]) # Pie Chart 2
ax3 = fig.add_subplot(gs[:2, 2]) # Rechts über 2 Zeilen
ax4 = fig.add_subplot(gs[1, :2]) # Tabelle
ax5 = fig.add_subplot(gs[2, :2]) # Kennzahlen-Zeile (bleibt)

# Generalist Pie Chart mit 3D-Effekt
wedges1, texts1, autotexts1 = ax1.pie(generalist, labels=categories,
autopct='%1.0f%%',

                                colors=colors_generalist,
                                explode=(0.08, 0.08, 0.08, 0.08, 0.08),
                                shadow=False, # Schatten entfernt
                                startangle=45,
                                wedgeprops={'edgecolor': 'black',
                                              'linewidth': 2,
                                              'linestyle': '-',
                                              'alpha': 0.9},
                                textprops={'fontsize': 9,
                                              'fontweight': 'bold',
                                              'color': 'white'})

ax1.set_title("Kunde C: Generalist\n(Gleichmäßige Diversifikation)",
              fontsize=12, fontweight='bold', pad=15)

# Spezialist Pie Chart mit 3D-Effekt
wedges2, texts2, autotexts2 = ax2.pie(specialist, labels=categories,
autopct='%1.0f%%',

                                colors=colors_specialist,
                                explode=(0.15, 0.03, 0.03, 0.03, 0.03),
                                shadow=False, # Schatten entfernt
                                startangle=45,
                                wedgeprops={'edgecolor': 'black',
                                              'linewidth': 3,
                                              'linestyle': '-',
                                              'alpha': 0.9},

```

```

        textprops={'fontsize': 9,
                    'fontweight': 'bold',
                    'color': 'white'})

ax2.set_title("Kunde D: Spezialist\n(Fokus auf Elektronik)",
              fontsize=12, fontweight='bold', pad=15)

# Extra weiße, fette Schrift für 3D-Effekt
for autotext in autotexts1:
    autotext.set_color('white')
    autotext.set_fontweight('bold')
    autotext.set_fontsize(11)
    autotext.set_path_effects([path_effects.withStroke(linewidth=3,
foreground='black')])

for autotext in autotexts2:
    autotext.set_color('white')
    autotext.set_fontweight('bold')
    autotext.set_fontsize(11)
    autotext.set_path_effects([path_effects.withStroke(linewidth=3,
foreground='black')])

# Analyse-Panel rechts mit 3D-Box
ax3.axis('off')

```

Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```

# 3D-artiger Hintergrund
ax3.add_patch(plt.Rectangle((0, 0), 1, 1, transform=ax3.transAxes,
                             facecolor='lightgray', alpha=0.3,
                             edgecolor='black', linewidth=2))

```

```

ax3.text(0.05, 0.95, 'Sortiments-Analyse', fontsize=14, fontweight='bold',
        transform=ax3.transAxes,
        bbox=dict(boxstyle="round,pad=0.3", facecolor='steelblue', alpha=0.7,
edgecolor='black'))

# Kunde C Statistiken mit 3D-Boxen
ax3.text(0.05, 0.85, 'Kunde C (Generalist):', fontsize=11, fontweight='bold',
        transform=ax3.transAxes, color='white',
        bbox=dict(boxstyle="round,pad=0.3", facecolor='#FF6B6B', alpha=0.8,
edgecolor='black'))

stats_c = ['• 5 aktive Kategorien', '• Diversitäts-Index: 1.00',
          '• Gleichmäßige Verteilung', '• Cross-Selling-Potenzial']
for i, stat in enumerate(stats_c):
    ax3.text(0.05, 0.78 - i*0.04, stat, fontsize=10, transform=ax3.transAxes,
            bbox=dict(boxstyle="round,pad=0.2", facecolor='white', alpha=0.8,
edgecolor='gray'))

# Kunde D Statistiken mit 3D-Boxen
ax3.text(0.05, 0.55, 'Kunde D (Spezialist):', fontsize=11, fontweight='bold',
        transform=ax3.transAxes, color='white',
        bbox=dict(boxstyle="round,pad=0.3", facecolor='#E74C3C', alpha=0.8,
edgecolor='black'))

stats_d = ['• 5 Kategorien, 1 dominant', '• Diversitäts-Index: 0.37',
          '• 80% Fokus auf Elektronik', '• Category-Management']
for i, stat in enumerate(stats_d):
    ax3.text(0.05, 0.48 - i*0.04, stat, fontsize=10, transform=ax3.transAxes,
            bbox=dict(boxstyle="round,pad=0.2", facecolor='white', alpha=0.8,
edgecolor='gray'))

# Strategische Implikationen - VERKÜRZT
ax3.text(0.05, 0.25, 'Strategische Implikationen:', fontsize=11,
fontweight='bold',

```

```

        transform=ax3.transAxes, color='white',
        bbox=dict(boxstyle="round,pad=0.3", facecolor='darkgreen', alpha=0.8,
edgecolor='black'))

ax3.text(0.05, 0.18, 'Verschiedene Strategien erfordern\ningepasste Ansätze',
fontSize=10,
        transform=ax3.transAxes, style='italic',
        bbox=dict(boxstyle="round,pad=0.2", facecolor='lightyellow',
alpha=0.8, edgecolor='orange'))

# 3D-Tabelle unten
ax4.axis('off')

```

Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```

# Tabellen-Daten
table_data = [
    ['Merkmal', 'Generalist (Kunde C)', 'Spezialist (Kunde D)'],
    ['Kategorien aktiv', '5 (alle gleichmäßig)', '5 (1 dominant)'],
    ['Konzentration', '20% pro Kategorie', '80% Elektronik'],
    ['Diversität', 'Hoch (1.00)', 'Niedrig (0.37)'],
    ['Marketing-Fokus', 'Cross-Selling', 'Category-Expertise']
]

# Tabelle mit 3D-Effekt
table = ax4.table(cellText=table_data[1:],
                  colLabels=table_data[0],
                  cellLoc='center',
                  loc='center',
                  bbox=[0, 0, 1, 1])

table.auto_set_font_size(False)

```



```

table.set_fontsize(9)
table.scale(1, 2.2)

# 3D-Header-Stil
for i in range(3):
    table[(0, i)].set_facecolor('#2C3E50')
    table[(0, i)].set_text_props(weight='bold', color='white')
    table[(0, i)].set_edgecolor('black')
    table[(0, i)].set_linewidth(2)

# 3D-Zeilen mit Farbverläufen
colors_rows = ['#ECF0F1', '#BDC3C7']
for i in range(1, 5):
    for j in range(3):
        table[(i, j)].set_facecolor(colors_rows[i % 2])
        table[(i, j)].set_edgecolor('black')
        table[(i, j)].set_linewidth(1)

# Kennzahlen-Ausgabe (BLEIBT WO SIE IST)
ax5.axis('off')

```

Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```

kennzahlen_text = f"""\SORTIMENTSANALYSE: Kunde C - Konzentration:
{max(generalist)}% (niedrig) | Kunde D - Konzentration: {max(specialist)}%
(sehr hoch) | Verhältnis Hauptkategorie:
{max(specialist)/max(generalist):.1f}:1"""

ax5.text(0.5, 0.5, kennzahlen_text, fontsize=10,
         transform=ax5.transAxes, ha='center', va='center',
         fontfamily='monospace', fontweight='bold',
         bbox=dict(boxstyle="round,pad=0.3", facecolor='gold',

```

```

alpha=0.8, edgecolor='black', linewidth=2))

# Titel HÖHER positioniert mit mehr Abstand
plt.suptitle('Sortimentsverhalten: Generalist vs. Spezialist',
             fontsize=18, fontweight='bold', y=0.98, # Höher positioniert
             bbox=dict(boxstyle="round,pad=0.5", facecolor='gold', alpha=0.8,
             edgecolor='black', linewidth=3))

plt.tight_layout()
plt.show()

```

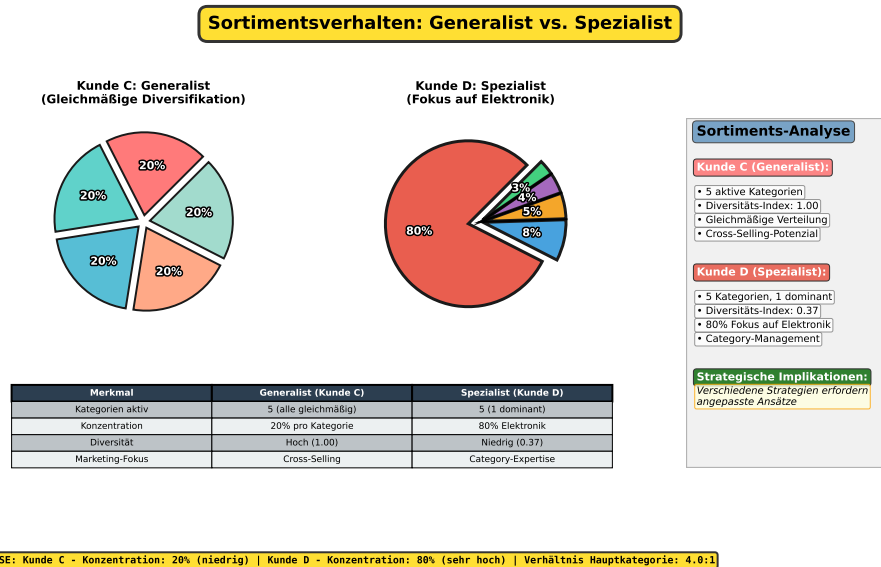


Abbildung A.6: Sortimentsverhalten: Vergleich von Generalisten (gleichmäßige Diversifikation) und Spezialisten (dominante Kategorie)

Kunde C verteilt seine Ausgaben gleichmäßig über verschiedene Kategorien, was sich in einem hohen Diversitätsindex widerspiegelt und Cross-Selling-Potenzial bietet. Kunde D hingegen fokussiert sich stark auf eine einzige Kategorie, was eine spezialisierte Ansprache im Category-Management erforderlich macht.

Anhang C: Datenaufbereitung und Voranalysen

Dieser Anhang dokumentiert die technischen Schritte, die den Haupttext flankieren. Die Abschnitte C.1 bis C.5 stützen die in Abschnitt 3.2 beschriebenen Datensätze sowie die praktische Aufbereitung in Abschnitt 5.1. Abschnitt C.6 liefert die Vorprüfung der Clustertendenz und bildet damit die Brücke zwischen Abschnitt 4.3 Validierung theoretisch und Abschnitt 5.2 Clustering Ergebnisse. Abschnitt C.7 enthält die Implementierung der in Abschnitt 4.2 beschriebenen Verfahren und erzeugt die in Abschnitt 5.2 ausgewerteten Clusterzuordnungen. Die inhaltliche Deutung der Segmente erfolgt in Abschnitt 5.3.

Der Anhang ist wie folgt gegliedert:

1. Variablenbeschreibung und Quellen (C.1)
2. Abbildung auf ein einheitliches Ereignisschema (C.2)
3. Bereinigungsverfahren und Qualitätsregeln (C.3)
4. Normalisierung der Zeitstempel und Kalendermerkmale (C.4)
5. Zusammenfassung des bereinigten Bestands und Plausibilisierung (C.5)
6. Clustertendenz mit der Hopkins-Statistik als Vorprüfung
7. Modellierungsschritte in Python für K-Means und HDBSCAN

In den Abschnitten C.1–C.5 wird mit R gearbeitet, um die Daten zu laden, bereinigen und in ein konsistentes Schema zu überführen. Ab C.6 kommen Python und spezialisierte Bibliotheken für Clustertendenz, Modellierung und Evaluation zum Einsatz. Diese Aufgabenteilung folgt der Praxis vieler Forschungsarbeiten: R für transparente Datenaufbereitung, Python für Machine Learning.

C.1 Variablenbeschreibung und Quellen

```
## eval=TRUE
## echo=TRUE
## include=TRUE
## results=TRUE
## message=FALSE
## warning=FALSE

# Pakete laden
library(readr)
library(dplyr)
library(here)

# Versionen dokumentieren
pkgs <- c("readr", "dplyr", "here")
vers <- sapply(pkgs, function(p) as.character(utils::packageVersion(p)))
attached <- pkgs %in% sub("^package:", "", search())
data.frame(package = pkgs, version = vers, attached = attached)
```

Terminal-Output

	package	version	attached
readr	readr	2.1.5	TRUE
dplyr	dplyr	1.1.4	TRUE
here	here	1.0.1	TRUE

```
# Pfade bauen
p_events <- here("data", "RR", "events.csv")
p_item1 <- here("data", "RR", "item_properties_part1.csv")
p_item2 <- here("data", "RR", "item_properties_part2.csv")
p_cat <- here("data", "RR", "category_tree.csv")
p_or2 <- if (file.exists(here("data", "OR2", "online_retail_II.csv"))) {
  here("data", "OR2", "online_retail_II.csv")
}
```

```

    } else {
      here("data", "OR2", "online_retail_11.csv")
    }

# Laden
events      <- read_csv(p_events, show_col_types = FALSE)
item_props_1 <- read_csv(p_item1,  show_col_types = FALSE)
item_props_2 <- read_csv(p_item2,  show_col_types = FALSE)
category_tree <- read_csv(p_cat,    show_col_types = FALSE)
online_retail <- read_csv(p_or2,    show_col_types = FALSE)

# Erste 5 Zeilen je Datensatz
head(events, 5)

```

 Terminal-Output

```

# A tibble: 5 x 5
  timestamp visitorid event itemid transactionid
  <dbl>      <dbl> <chr>  <dbl>      <dbl>
1 1433221332117 257597 view  355908      NA
2 1433224214164 992329 view  248676      NA
3 1433221999827 111016 view  318965      NA
4 1433221955914 483717 view  253185      NA
5 1433221337106 951259 view  367447      NA

```

```
head(item_props_1, 5)
```

 Terminal-Output

```

# A tibble: 5 x 4
  timestamp itemid property value
  <dbl>    <dbl> <chr>    <chr>
1 1435460400000 460429 categoryid 1338
2 1441508400000 206783 888      1116713 960601 n277.200
3 1439089200000 395014 400      n552.000 639502 n720.000 424566
4 1431226800000 59481 790      n15360.000
5 1431831600000 156781 917      828513

```

```
head(item_props_2, 5)
```

Terminal-Output

```
# A tibble: 5 x 4
  timestamp itemid property value
  <dbl>    <dbl> <chr>    <chr>
1 1433041200000 183478 561      769062
2 1439694000000 132256 976      n26.400 1135780
3 1435460400000 420307 921      1149317 1257525
4 1431831600000 403324 917      1204143
5 1435460400000 230701 521      769062
```

```
head(category_tree, 5)
```

Terminal-Output

```
# A tibble: 5 x 2
  categoryid parentid
  <dbl>    <dbl>
1    1016      213
2     809      169
3     570        9
4    1691      885
5     536     1691
```

```
head(online_retail, 5)
```

Terminal-Output

```
# A tibble: 5 x 8
  Invoice StockCode Description Quantity InvoiceDate Price `Customer ID
  <chr>   <chr>    <chr>         <dbl> <dtm>    <dbl>      <dbl>
1 489434  85048    "15CM CHRI~         12 2009-12-
01 07:45:00 6.95      13085
2 489434  79323P    "PINK CHER~         12 2009-12-
01 07:45:00 6.75      13085
3 489434  79323W    "WHITE CHE~         12 2009-12-
01 07:45:00 6.75      13085
4 489434  22041    "RECORD FR~         48 2009-12-
01 07:45:00 2.1       13085
5 489434  21232    "STRAWBERR~         24 2009-12-
01 07:45:00 1.25      13085
```



```
Terminal-Output
# i 1 more variable: Country <chr>
```

Die Ausgaben verdeutlichen die Struktur der Datensätze. Zur besseren Übersicht werden die wichtigsten Variablen und ihre Bedeutungen zusätzlich tabellarisch dargestellt:

Tabelle 7.1: Übersicht der Variablen in beiden Datenquellen

Variable	Quelle	Bedeutung
<code>timestamp</code>	RetailRocket (events)	Zeitstempel in Millisekunden seit 1970, Angabe des Ereigniszeitpunkts
<code>visitorid</code>	RetailRocket (events)	Anonymisierte Kennung eines Besuchers
<code>event</code>	RetailRocket (events)	Typ des Ereignisses: <i>view</i> , <i>addtocart</i> , <i>transaction</i>
<code>itemid</code>	RetailRocket (events)	Artikelkennung
<code>transactionid</code>	RetailRocket (events)	ID der Transaktion (optional, nur bei Käufen)
<code>property</code>	RetailRocket (item_properties)	Attribut eines Artikels (z. B. Kategorie, Preis, technische Merkmale)
<code>value</code>	RetailRocket (item_properties)	Wert des jeweiligen Attributs
<code>categoryid</code>	RetailRocket (category_tree)	Identifikator einer Kategorie
<code>parentid</code>	RetailRocket (category_tree)	Übergeordnete Kategorie, beschreibt die Hierarchie
<code>Invoice</code>	Online Retail II	Rechnungsnummer
<code>StockCode</code>	Online Retail II	Artikelcode

Variable	Quelle	Bedeutung
Description	Online Retail II	Kurzbeschreibung des Artikels
Quantity	Online Retail II	Menge der verkauften Artikel
InvoiceDate	Online Retail II	Zeitstempel der Rechnungsstellung
Price	Online Retail II	Einzelpreis pro Artikel
Customer ID	Online Retail II	Anonymisierte Kundenkennung
Country	Online Retail II	Land des Kunden

Diese Gegenüberstellung verknüpft die in 3.2 beschriebenen Quellen mit den nachfolgenden Transformationsschritten.

C.2: Abbildung auf ein einheitliches Ereignisschema

Ziel ist ein gemeinsames Schema für beide Quellen. RetailRocket liefert Interaktionen, Online Retail II liefert Käufe. Das vereinheitlichte Ereignisschema ermöglicht konsistente Merkmalsbildung für 4.1 und robuste Inputs für 4.2–4.3.

Zuordnung der Quell- zu den Zielvariablen

Tabelle 7.2: Abbildung der Variablen auf das Ereignisschema

	RetailRocket	Online Retail	
Zielvariable	(Quelle)	II (Quelle)	Bedeutung
customer_id	visitorid	Customer ID	Anonymisierte Kennung des Kunden
item_id	itemid	StockCode	Artikelkennung
timestamp	timestamp (ms seit 1970)	InvoiceDate	Zeitpunkt des Ereignisses / der Transaktion
event_type	event (<i>view</i> , <i>addtocart</i> , <i>transaction</i>)	implizit: immer <i>purchase</i>	Typ des Ereignisses

	RetailRocket	Online Retail	
Zielvariable	(Quelle)	II (Quelle)	Bedeutung
transactionid	transactionid (falls Kauf)	Invoice	Eindeutige Transaktionskennung
quantity	nicht enthalten	Quantity	Anzahl der verkauften Artikel
price	über item_properties	Price	Einzelpreis pro Artikel
revenue	berechnet: quantity × price	berechnet: Quantity × Price	Umsatz pro Ereignis
category	aus category_tree	ggf. aus Description	Zuordnung zu einer Produktkategorie

Die Abbildung in ein gemeinsames Schema stellt sicher, dass spätere Merkmalsberechnungen (z. B. Recency, Frequency, Monetary-Werte, Kaufintervalle, Diversität) auf konsistenter Datenbasis erfolgen können.

Die Umsetzung der Transformation erfolgt mit R. Der folgende Chunk zeigt exemplarisch, wie die Zielvariablen aus beiden Quellen generiert werden.

```

## eval: TRUE
## echo: TRUE
## include: TRUE
## results: TRUE
## message: FALSE
## warning: FALSE

library(dplyr)
library(lubridate)

# RetailRocket in UTC einlesen, dann auf Europe/London abbilden

```

```
rr_events <- events %>%
  transmute(
    customer_id = visitorid,
    item_id     = itemid,
    timestamp   = as.POSIXct(timestamp/1000, origin = "1970-01-01", tz =
"UTC"),
    event_type  = event,
    transactionid = transactionid
  ) %>%
  mutate(timestamp = with_tz(timestamp, "Europe/London"),
    quantity = NA_real_,
    price    = NA_real_,
    revenue  = NA_real_,
    category = NA_character_)

# Online Retail II: lokale Zeiten explizit als Europe/London interpretieren
or2_events <- online_retail %>%
  transmute(
    transactionid = Invoice,
    item_id       = StockCode,
    customer_id   = `Customer ID`,
    timestamp     = force_tz(InvoiceDate, "Europe/London"),
    quantity      = Quantity,
    price         = Price,
    country       = Country,
    description   = Description,
    event_type    = "purchase",
    revenue       = Quantity * Price,
    category      = Description
  )

head(rr_events, 5)
```

```

Terminal-Output

# A tibble: 5 x 9
  customer_id item_id timestamp          event_type transactionid quantity
  <dbl>      <dbl> <dtm>          <chr>          <dbl>      <dbl>
1    257597    355908 2015-06-02 06:02:12 view              NA         NA
2    992329    248676 2015-06-02 06:50:14 view              NA         NA
3    111016    318965 2015-06-02 06:13:19 view              NA         NA
4    483717    253185 2015-06-02 06:12:35 view              NA         NA
5    951259    367447 2015-06-02 06:02:17 view              NA         NA
# i 3 more variables: price <dbl>, revenue <dbl>, category <chr>

```

```
head(or2_events, 5)
```

```

Terminal-Output

# A tibble: 5 x 11
  transactionid item_id customer_id timestamp          quantity price country
  <chr>         <chr>      <dbl> <dtm>          <dbl> <dbl> <chr>
1 489434         85048      13085 2009-12-01 07:45:00      12  6.95 United K
2 489434         79323P     13085 2009-12-01 07:45:00      12  6.75 United K
3 489434         79323W     13085 2009-12-01 07:45:00      12  6.75 United K
4 489434         22041     13085 2009-12-01 07:45:00      48  2.1  United K
5 489434         21232     13085 2009-12-01 07:45:00      24  1.25 United K
# i 4 more variables: description <chr>, event_type <chr>, revenue <dbl>,
#   category <chr>

```

In der harmonisierten Tabelle von RetailRocket bleiben die Variablen **quantity**, **price**, **revenue** und **category** zunächst leer, da diese Informationen im Rohdatensatz nicht in direkter Form vorliegen. Diese Spalten wurden dennoch angelegt, um die Struktur beider Quellen konsistent zu halten und spätere Ergänzungen zu ermöglichen.

Bei Online Retail II sind diese Variablen hingegen vollständig verfügbar: Transaktionskennungen, Mengen, Preise und Umsätze können direkt übernommen werden. Der Ereignistyp ist dort implizit immer ein Kauf (*purchase*), sodass keine Unterscheidung zwischen verschiedenen Interaktionen möglich ist.

Die Abbildung verdeutlicht damit die komplementäre Natur der Daten: RetailRocket liefert eine feingranulare Sicht auf Nutzerverhalten, Online Retail II eine wertbasierte Sicht auf Käufe. Das vereinheitlichte Ereignisschema stellt sicher, dass in den folgenden Schritten (vgl. C.3 Bereinigungsverfahren und C.4 Normalisierung der

Zeitstempel) eine konsistente Datenbasis für die Merkmalsbildung geschaffen wird.

C.3: Bereinigungsverfahren

Die Rohdaten enthalten neben den relevanten Informationen auch fehlerhafte oder problematische Einträge. Um eine konsistente Datenbasis für die spätere Merkmalsbildung zu schaffen, wurden die folgenden Regeln angewandt:

1. Pflichtfelder: Beobachtungen ohne Kunden-, Artikel- oder Zeitangabe wurden entfernt.
2. Duplikate: Exakte Dubletten (gleiche Kunden-, Artikel-, Zeit- und Ereignisinformationen) wurden eliminiert.
3. Rückgaben: Im Datensatz Online Retail II wurden negative Mengen sowie Rechnungsnummern mit Präfix „C“ als Rückgaben gekennzeichnet. Diese Ereignisse erhielten den Typ `return`.
4. Ausreißer: Mengen und Preise wurden per Winsorisierung auf das 1. und 99. Perzentil begrenzt, um extreme Werte abzuflachen.
5. Fehlerhafte Werte: Offensichtlich ungültige Beobachtungen (z. B. Mengen 0 nach Winsorisierung) wurden ausgeschlossen.

Im RetailRocket-Datensatz bleiben die Variablen `quantity`, `price` und `revenue` leer, da diese Informationen dort nicht in direkter Form vorliegen. Die Ereignisse konzentrieren sich auf Interaktionen wie *view*, *addtocart* und *transaction*. Im Online-Retail-II-Datensatz konnten dagegen Transaktionskennungen, Mengen, Preise und Umsätze bereinigt und in plausibler Form übernommen werden.

Die Anwendung dieser Regeln stellt sicher, dass Auswertungen auf stabilen Grundlagen erfolgen und Verzerrungen durch fehlerhafte Einträge vermieden werden. Die bereinigten Daten bilden damit die Ausgangsbasis für die Normalisierung der Zeitangaben in Abschnitt C.4.

Das Recency-Merkmal wird nicht im Preprocessing berechnet, sondern erst beim Feature Engineering auf Kundenebene in Abschnitt C.7.

```

# benötigte Pakete
library(dplyr)
library(stringr)

if (!exists("rr_events")) stop("rr_events fehlt. Bitte C.2 ausführen.")
if (!exists("or2_events")) stop("or2_events fehlt. Bitte C.2 ausführen.")

safe_winsorize <- function(x, p_low = 0.01, p_high = 0.99) {
  if (!is.numeric(x)) return(x)
  nn <- sum(!is.na(x))
  if (nn < 10) return(x)
  lo <- suppressWarnings(quantile(x, probs = p_low, na.rm = TRUE))
  hi <- suppressWarnings(quantile(x, probs = p_high, na.rm = TRUE))
  as.numeric(pmin(pmax(x, lo), hi))
}

# RetailRocket bereinigen
rr_clean <- rr_events %>%
  select(customer_id, item_id, timestamp, event_type, transactionid,
         quantity, price, revenue, category) %>%
  filter(!is.na(customer_id), !is.na(item_id), !is.na(timestamp),
         !is.na(event_type)) %>%
  distinct(customer_id, item_id, timestamp, event_type, transactionid,
           .keep_all = TRUE)

# Online Retail II: Rückgaben sauber behandeln
or2_clean <- or2_events %>%
  mutate(
    return_flag = (!is.na(quantity) & quantity < 0) |
                  (!is.na(transactionid) &
str_starts(as.character(transactionid), "C")),
    event_type = if_else(return_flag, "return", "purchase"),
    quantity_abs = if_else(!is.na(quantity), abs(quantity), NA_real_),
    revenue_signed = quantity * price,

```

```

revenue_abs = quantity_abs * price,
category    = coalesce(category, description)
) %>%
mutate(
  quantity_w = safe_winsorize(quantity_abs, 0.01, 0.99),
  price_w    = safe_winsorize(price,      0.01, 0.99),
  revenue_w  = quantity_w * price_w
) %>%
select(transactionid, item_id, customer_id, timestamp,
        quantity_abs, quantity_w, price, price_w, event_type,
        revenue_signed, revenue_abs, revenue_w,
        category, country, description) %>%
filter(is.na(quantity_w) | quantity_w > 0,
       is.na(price_w)    | price_w    > 0)

head(rr_clean, 5)

```

 Terminal-Output

```

# A tibble: 5 x 9
  customer_id item_id timestamp          event_type transactionid quantity
  <dbl>      <dbl> <dtm>          <chr>          <dbl>      <dbl>
1    257597   355908 2015-06-02 06:02:12 view             NA         NA
2    992329   248676 2015-06-02 06:50:14 view             NA         NA
3    111016   318965 2015-06-02 06:13:19 view             NA         NA
4    483717   253185 2015-06-02 06:12:35 view             NA         NA
5     951259   367447 2015-06-02 06:02:17 view             NA         NA
# i 3 more variables: price <dbl>, revenue <dbl>, category <chr>

```

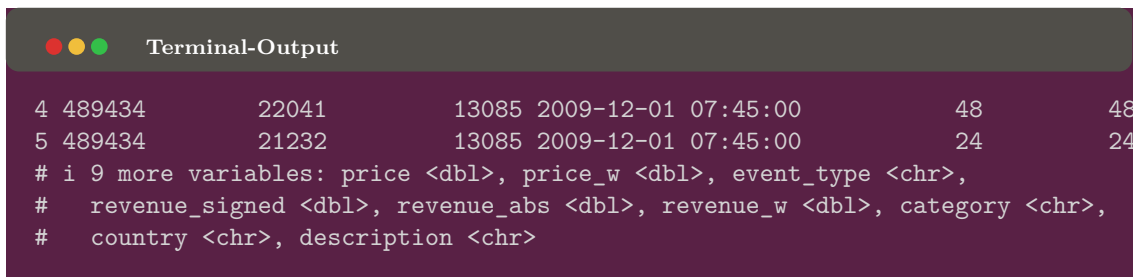
```
head(or2_clean, 5)
```

 Terminal-Output

```

# A tibble: 5 x 15
  transactionid item_id customer_id timestamp          quantity_abs quantity_w
  <chr>          <chr>      <dbl> <dtm>          <dbl>      <dbl>
1 489434        85048    13085 2009-12-01 07:45:00      12         12
2 489434        79323P    13085 2009-12-01 07:45:00      12         12
3 489434        79323W    13085 2009-12-01 07:45:00      12         12

```



```

Terminal-Output
4 489434      22041      13085 2009-12-01 07:45:00      48      48
5 489434      21232      13085 2009-12-01 07:45:00      24      24
# i 9 more variables: price <dbl>, price_w <dbl>, event_type <chr>,
#   revenue_signed <dbl>, revenue_abs <dbl>, revenue_w <dbl>, category <chr>,
#   country <chr>, description <chr>

```

Die Vorschau zeigt, dass Pflichtfelder belegt sind, Duplikate entfernt wurden und Rückgaben in Online Retail II über `event_type = return` gekennzeichnet sind. Mengen und Preise wurden dort per Winsorisierung begrenzt und als `quantity_w` und `price_w` bereitgestellt.

C.4: Normalisierung der Zeitstempel

Neben inhaltlichen Bereinigungen ist auch die einheitliche Behandlung der Zeitangaben erforderlich. Beide Datensätze enthalten Zeitinformationen, die jedoch in unterschiedlicher Form vorliegen.

- Im RetailRocket-Datensatz werden Zeitstempel in Millisekunden seit dem 1. Januar 1970 gespeichert
- Im Online-Retail-II-Datensatz liegt das Kaufdatum bereits als Datumsfeld (`InvoiceDate`) vor, allerdings ohne einheitliche Ableitung von Kalendermerkmalen

Um die Daten konsistent nutzen zu können, wurden folgende Schritte durchgeführt:

1. Konvertierung in ein einheitliches Format. Alle Zeitangaben wurden in das POSIXct-Format überführt und zusätzlich als Datum (`date`) abgespeichert.
2. Ableitung von Kalendermerkmalen. Aus den Zeitstempeln wurden standardisierte Variablen für Wochentag (`wday`), Stunde (`hour`), Monat (`month`) und Jahr (`year`) berechnet.
3. Harmonisierung der Skalen. Damit liegen beide Quellen auf derselben Zeitskala vor, was aggregierte Analysen und Vergleiche zwischen RetailRocket und Online Retail II ermöglicht.

Die Normalisierung erlaubt es, zeitliche Muster in beiden Quellen konsistent zu

erfassen, beispielsweise bevorzugte Kaufzeiten oder saisonale Effekte. Diese Variablen werden in der Merkmalsbildung (vgl. Abschnitt) eingesetzt, um Kundencluster auch anhand von Verhaltensrhythmen zu unterscheiden.

```
## include: true
library(reticulate)
knitr::opts_chunk$set(python.reticulate = TRUE)
set.seed(42)
```

```
# Dateien:
#   cache/rr_time.rds
#   cache/or2_time.rds

## eval: true
## echo: true
## message: false
## warning: false

library(dplyr)
library(lubridate)

if (!exists("rr_clean") || !exists("or2_clean")) {
  stop("C.4 benötigt rr_clean und or2_clean aus C.3.")
}

dir.create("cache", showWarnings = FALSE)
rr_file <- "cache/rr_time.rds"
or2_file <- "cache/or2_time.rds"

to_iso_utc <- function(x) {
  format(with_tz(as.POSIXct(x, tz = "Europe/London"), "UTC"),
    "%Y-%m-%dT%H:%M:%SZ")
}
```



```
if (file.exists(rr_file) && file.exists(or2_file)) {
  rr_time <- readRDS(rr_file)
  or2_time <- readRDS(or2_file)
} else {
  rr_time <- rr_clean %>%
    mutate(
      timestamp = with_tz(as.POSIXct(timestamp, tz = "Europe/London"),
"Europe/London"),
      date = as.Date(timestamp),
      wday = wday(timestamp, label = TRUE, abbr = TRUE, week_start =
1),
      hour = hour(timestamp),
      month = month(timestamp, label = TRUE, abbr = TRUE),
      year = year(timestamp),
      timestamp_iso = to_iso_utc(timestamp)
    )

  or2_time <- or2_clean %>%
    mutate(
      timestamp = with_tz(as.POSIXct(timestamp, tz = "Europe/London"),
"Europe/London"),
      date = as.Date(timestamp),
      wday = wday(timestamp, label = TRUE, abbr = TRUE, week_start =
1),
      hour = hour(timestamp),
      month = month(timestamp, label = TRUE, abbr = TRUE),
      year = year(timestamp),
      timestamp_iso = to_iso_utc(timestamp)
    )

  saveRDS(rr_time, rr_file)
  saveRDS(or2_time, or2_file)
}
```

```

# Python-Views: Käufe und Rückgaben getrennt übergeben
py$rr_time <- rr_time %>%
  select(customer_id, item_id, timestamp = timestamp_iso, event_type)

py$or2_purchases <- or2_time %>%
  filter(event_type == "purchase") %>%
  select(transactionid, customer_id, item_id, timestamp = timestamp_iso,
    quantity_w, price_w, revenue_w, category)

py$or2_returns <- or2_time %>%
  filter(event_type == "return") %>%
  select(transactionid, customer_id, timestamp = timestamp_iso,
    revenue_signed, revenue_w) # <- beide Werte mitgeben

list(
  rr_time_head = head(select(rr_time, customer_id, item_id, timestamp,
    timestamp_iso, event_type), 3),
  or2_pur_head = head(py$or2_purchases, 3),
  or2_ret_head = head(py$or2_returns, 3)
)

```

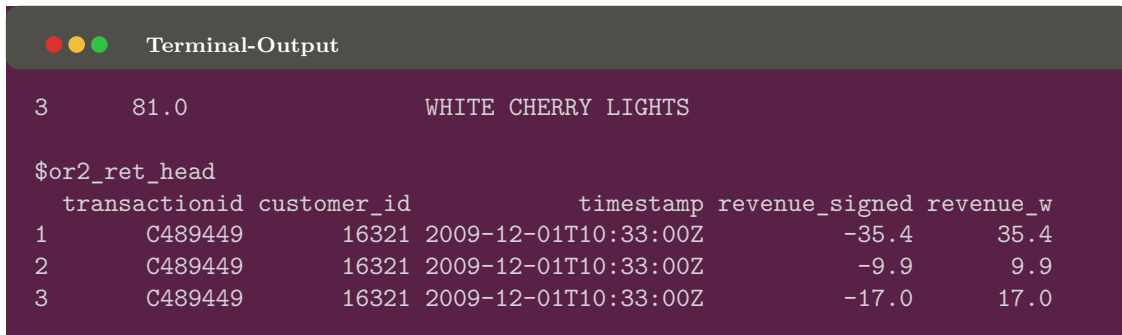
Terminal-Output

```

$rr_time_head
# A tibble: 3 x 5
  customer_id item_id timestamp          timestamp_iso event_type
  <dbl>    <dbl> <dtm>          <chr>          <chr>
1    257597  355908 2015-06-02 06:02:12 2015-06-02T05:02:12Z view
2    992329  248676 2015-06-02 06:50:14 2015-06-02T05:50:14Z view
3    111016  318965 2015-06-02 06:13:19 2015-06-02T05:13:19Z view

$or2_pur_head
  transactionid customer_id item_id          timestamp quantity_w price_w
1    489434      13085   85048 2009-12-01T07:45:00Z         12   6.95
2    489434      13085  79323P 2009-12-01T07:45:00Z         12   6.75
3    489434      13085  79323W 2009-12-01T07:45:00Z         12   6.75
  revenue_w category
1    83.4 15CM CHRISTMAS GLASS BALL 20 LIGHTS
2    81.0          PINK CHERRY LIGHTS

```



Terminal-Output

```

3      81.0      WHITE CHERRY LIGHTS

$or2_ret_head
  transactionid customer_id      timestamp revenue_signed revenue_w
1      C489449      16321 2009-12-01T10:33:00Z      -35.4      35.4
2      C489449      16321 2009-12-01T10:33:00Z      -9.9      9.9
3      C489449      16321 2009-12-01T10:33:00Z     -17.0     17.0

```

Die Normalisierung liegt damit für beide Datensätze in identischer Struktur vor. Die abgeleiteten Kalendermerkmale werden in der Merkmalsbildung genutzt und erlauben zeitliche Musteranalysen ohne zusätzliche Transformationen.

C.5: Zusammenfassung des bereinigten Datenbestands

Die Abschnitte C.3 und C.4 haben die Bereinigung und die Normalisierung der Zeitangaben dokumentiert. In diesem Abschnitt wird der bereinigte Datenbestand in komprimierter Form zusammengefasst. Dadurch lässt sich die Qualität der Datenbasis einschätzen, die in Abschnitt für die Ableitung der Cluster-Merkmale genutzt wird.

Zentrale Fragen der Plausibilisierung sind:

1. Wie groß sind die Datensätze nach der Bereinigung?
2. Welche Verteilungen zeigen die kaufbezogenen Variablen in Online Retail II?
3. Sind die Strukturen der beiden Quellen konsistent nutzbar?

```

#| eval: true
#| echo: true
#| message: false
#| warning: false
#| dependson: ["C.4-zeit-normalisierung"]

library(dplyr)
library(tidyr)
library(lubridate)

```

```

if (!exists("rr_time") || !exists("or2_time")) stop("C.5 benötigt rr_time und
or2_time aus C.4.")
file_sum <- "cache/zusammenfassung.rds"

if (file.exists(file_sum)) {
  res <- readRDS(file_sum)
} else {
  dim_summary <- tibble::tibble(
    datensatz = c("RetailRocket (bereinigt + normalisiert)", "Online Retail II
(bereinigt + normalisiert)"),
    n_zeilen = c(nrow(rr_time), nrow(or2_time)),
    n_spalten = c(ncol(rr_time), ncol(or2_time))
  )

  missing_rr <- rr_time %>% summarise(across(c(customer_id, item_id,
timestamp, event_type), ~ sum(is.na(.)))) %>%
    pivot_longer(everything(), names_to = "feld", values_to = "missing") %>%
mutate(datensatz = "RetailRocket")

  missing_or2 <- or2_time %>% summarise(across(c(customer_id, item_id,
timestamp, event_type, quantity_w, price_w, revenue_w), ~ sum(is.na(.)))) %>%
    pivot_longer(everything(), names_to = "feld", values_to = "missing") %>%
mutate(datensatz = "Online Retail II")

  duplicate_summary <- bind_rows(
    rr_time %>% mutate(key = paste(customer_id, item_id, timestamp,
event_type, sep = "|")) %>%
      summarise(dups = sum(duplicated(key)), share =
round(mean(duplicated(key)), 6)) %>% mutate(datensatz = "RetailRocket"),
    or2_time %>% mutate(key = paste(customer_id, item_id, timestamp,
event_type, sep = "|")) %>%
      summarise(dups = sum(duplicated(key)), share =
round(mean(duplicated(key)), 6)) %>% mutate(datensatz = "Online Retail II")
  ) %>% relocate(datensatz)

```

```

quantiles <- c(0, .01, .05, .25, .5, .75, .95, .99, 1)

dist_or2 <- or2_time %>%
  filter(event_type == "purchase") %>%                                #
nur Käufe
  select(quantity_w, price_w, revenue_w) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "wert") %>%
  filter(is.finite(wert)) %>%
  group_by(variable) %>%
  summarise(
    n=n(), missing=sum(is.na(wert)),
    mean=mean(wert,na.rm=TRUE), sd=sd(wert,na.rm=TRUE),
    q0=quantile(wert,0,na.rm=TRUE), q01=quantile(wert,.01,na.rm=TRUE),
    q05=quantile(wert,.05,na.rm=TRUE), q25=quantile(wert,.25,na.rm=TRUE),
    q50=quantile(wert,.5,na.rm=TRUE), q75=quantile(wert,.75,na.rm=TRUE),
    q95=quantile(wert,.95,na.rm=TRUE), q99=quantile(wert,.99,na.rm=TRUE),
    q100=quantile(wert,1,na.rm=TRUE), .groups="drop"
  )

event_summary <- bind_rows(
  rr_time %>% count(event_type, name = "n") %>% mutate(datensatz =
"RetailRocket"),
  or2_time %>% count(event_type, name = "n") %>% mutate(datensatz = "Online
Retail II")
) %>% relocate(datensatz)

coverage_summary <- bind_rows(
  rr_time %>% mutate(year_month = format(as.Date(timestamp), "%Y-%m")) %>%
    count(year_month, name = "n") %>% mutate(datensatz = "RetailRocket"),
  or2_time %>% mutate(year_month = format(as.Date(timestamp), "%Y-%m")) %>%
    count(year_month, name = "n") %>% mutate(datensatz = "Online Retail II")
) %>% arrange(datensatz, year_month)

```

```

res <- list(
  dimensionen      = dim_summary,
  fehlende_werte   = bind_rows(missing_rr, missing_or2) %>%
relocate(datensatz),
  duplikate        = duplicate_summary,
  verteilungen_or2 = dist_or2,
  ereignisse       = event_summary,
  zeitliche_abdeckung = coverage_summary
)

saveRDS(res, file_sum, compress = "xz")
}

res

```

Terminal-Output

```

$dimensionen
# A tibble: 2 x 3
  datensatz                                n_zeilen n_spalten
  <chr>                                <int>    <int>
1 RetailRocket (bereinigt + normalisiert) 2755641      15
2 Online Retail II (bereinigt + normalisiert) 1067371      21

$fehlende_werte
# A tibble: 11 x 3
  datensatz      feld      missing
  <chr>      <chr>    <int>
1 RetailRocket customer_id      0
2 RetailRocket item_id      0
3 RetailRocket timestamp      0
4 RetailRocket event_type      0
5 Online Retail II customer_id 243007
6 Online Retail II item_id      0
7 Online Retail II timestamp      0
8 Online Retail II event_type      0
9 Online Retail II quantity_w      0
10 Online Retail II price_w      0
11 Online Retail II revenue_w      0

$duplikate
# A tibble: 2 x 3

```

```

Terminal-Output

  datensatz      dups  share
  <chr>         <int> <dbl>
1 RetailRocket      0  0
2 Online Retail II 47928 0.0449

$verteilungen_or2
# A tibble: 3 x 14
  variable      n missing  mean    sd    q0    q01    q05    q25    q50    q75    q9
  <chr>      <int>   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 price_w  1.04e6      0  3.29  3.30  0.21  0.25  0.42  1.25  2.1  4.13  9.9
2 quantity~ 1.04e6      0  9.03 16.7    1    1    1    1    3   10   30
3 revenue_w 1.04e6      0 17.3 35.7  0.21  0.55  1.25  3.9  9.96 17.7 54.2
# i 2 more variables: q99 <dbl>, q100 <dbl>

$ereignisse
# A tibble: 5 x 3
  datensatz      event_type      n
  <chr>         <chr>      <int>
1 RetailRocket addtocart    68966
2 RetailRocket transaction 22457
3 RetailRocket view       2664218
4 Online Retail II purchase 1044420
5 Online Retail II return   22951

$zeitliche_abdeckung
# A tibble: 30 x 3
  year_month      n datensatz
  <chr>      <int> <chr>
1 2009-12   45228 Online Retail II
2 2010-01   31555 Online Retail II
3 2010-02   29388 Online Retail II
4 2010-03   41511 Online Retail II
5 2010-04   34057 Online Retail II
6 2010-05   35323 Online Retail II
7 2010-06   39983 Online Retail II
8 2010-07   33383 Online Retail II
9 2010-08   33306 Online Retail II
10 2010-09   42091 Online Retail II
# i 20 more rows

```

Die Übersicht zeigt, dass beide Quellen nach der Bereinigung eine substanzielle Anzahl an Beobachtungen enthalten und somit ausreichend groß für die geplante Clusteranalyse sind.

Die kaufbezogenen Variablen im Online-Retail-II-Datensatz weisen nach Winsorisierung plausible Wertebereiche auf, extreme Ausreißer wurden erfolgreich begrenzt.

Rückgaben sind als eigene Ereignisse gekennzeichnet und können später gesondert berücksichtigt werden.

Der RetailRocket-Datensatz bildet dagegen ausschließlich Interaktionen ab, liefert aber mit den normalisierten Zeitfeldern eine konsistente Ergänzung zur transaktions-orientierten Sicht von Online Retail II.

Damit ist eine robuste und vergleichbare Datenbasis geschaffen, auf der in Abschnitt die eigentlichen Analysemerkmale konstruiert werden.

C.6 Clustertendenz mit der Hopkins-Statistik

Ziel dieses Abschnitts ist eine transparente Vorprüfung, ob der gewählte Merkmalsraum eine ausgeprägte Tendenz zur Clusterbildung zeigt. Die Hopkins Statistik dient hier als heuristischer Pre Check und überbrückt damit zwischen Abschnitt 4.3 Validierung theoretisch und Abschnitt 5.2 Clustering Ergebnisse. Wichtig für die Interpretation ist die Einordnung der Werte:

- H etwa 0.5 deutet auf eine eher zufällige Verteilung hin.
- H deutlich über 0.5 spricht für Clustertendenz.
- H deutlich unter 0.5 weist auf regelmäßige anti clusternde Muster hin.

Die Analyse erfolgt reproduzierbar mit Stichprobenziehung, Mehrfachschätzung und Konfidenzintervall. Große Rechenlast wird durch Subsampling gesteuert. Alle Berechnungen basieren auf den in C.4 normalisierten Daten `or2_time` und verwenden die kaufrelevanten Merkmale `quantity w`, `price w` und `revenue w`.

```
#!/ eval: true
#!/ echo: true
#!/ message: false
#!/ warning: false
#!/ dependson: ["C.4-zeit-normalisierung"]

library(dplyr)
library(tibble)
library(clustertend)
```



```

if (!exists("or2_time")) stop("C.6 benötigt or2_time aus C.4.")
if (!exists("FAST")) FAST <- TRUE

dir.create("cache", showWarnings = FALSE)
file_hop <- "cache/hopkins.rds"

if (file.exists(file_hop)) {
  hop_res <- readRDS(file_hop)
} else {
  set.seed(42)
  HOP_NROWS <- if (FAST) 1500L else 5000L
  HOP_R <- if (FAST) 15L else 40L
  HOP_N <- 300L

  dat0 <- or2_time %>%
    filter(event_type == "purchase") %>%           # nur Käufe für
    Clustertendenz
    select(quantity_w, price_w, revenue_w) %>%
    filter(if_all(everything(), ~ is.finite(.)))
  stopifnot(nrow(dat0) >= 50)

  dat_s <- dplyr::slice_sample(dat0, n = min(HOP_NROWS, nrow(dat0)))
  X <- scale(as.matrix(dat_s))
  n_h <- min(HOP_N, nrow(X) - 1L)

  get_H <- function(mat, n) {
    res <- clustertend::hopkins(mat, n = n)
    if (is.list(res) && !is.null(res$H)) return(unname(res$H))
    as.numeric(res)[1]
  }

  h_vals <- replicate(HOP_R, get_H(X, n_h))
  H_mean <- mean(h_vals); H_med <- median(h_vals); H_sd <- stats::sd(h_vals)

```

```

H_ci    <- c(H_mean - 1.96 * H_sd / sqrt(HOP_R),
            H_mean + 1.96 * H_sd / sqrt(HOP_R))

interpretation <- dplyr::case_when(
  H_med > 0.75 ~ "deutliche Clustertendenz",
  H_med < 0.25 ~ "regelmäßige Struktur",
  TRUE       ~ "eher zufällig verteilt"
)

hop_res <- tibble(
  n_verfuegbar = nrow(dat0),
  n_verwendet  = nrow(X),
  hopkins_n    = n_h,
  repeats      = HOP_R,
  H_mean       = round(H_mean, 3),
  H_median     = round(H_med, 3),
  H_sd         = round(H_sd, 3),
  H_ci_low     = round(H_ci[1], 3),
  H_ci_up      = round(H_ci[2], 3),
  einordnung   = interpretation
)

saveRDS(hop_res, file_hop, compress = "xz")
}

hop_res

```

```

Terminal-Output

# A tibble: 1 x 10
  n_verfuegbar n_verwendet hopkins_n repeats H_mean H_median H_sd H_ci_low
      <int>      <int>      <int>  <int>  <dbl>   <dbl> <dbl> <dbl>
1    1044420      1500        300    15  0.016   0.014 0.005  0.013
# i 2 more variables: H_ci_up <dbl>, einordnung <chr>

```

Die Hopkins-Vorprüfung zeigt, ob der Datensatz eine hinreichende Clustertendenz aufweist und rechtfertigt damit die nachgelagerte Segmentbildung. Konzeptionell ist die Maßzahl in Abschnitt 4.3 Validierung verankert; praktisch dient sie als Vorstufe für die in Abschnitt 5.2 berichteten Clustering-Ergebnisse.

C.7 Modellierungsschritte mit Python: K-Means und HDBSCAN auf Kundenebene

Die eigentliche Clusteranalyse erfolgt auf aggregierten Kundenmerkmalen. Grundlage sind die in C.4 erzeugten `or2_purchases` und `or2_returns`.

Pro Kunde werden Recency, Frequency, Monetary (RFM) sowie Warenkorb- und Sortimentsmerkmale gebildet. Diese Features bilden die Basis für K-Means (zentroidbasiert) und HDBSCAN (dichtebasiert).

Damit die Ergebnisse reproduzierbar sind und nicht bei jedem Rendern neu gerechnet werden müssen, werden sie als `.rds` Dateien zwischengespeichert.

Die Bewertung der Cluster erfolgt später in Kapitel 5 anhand von Silhouette, Davies–Bouldin und DBCV.

```
#!/ eval: true
#!/ echo: true
#!/ message: false
#!/ warning: false
#!/ dependson: ["C.4-zeit-normalisierung"]

library(dplyr)
library(lubridate)
library(tidyr)
library(purrr)

cust_file <- "cache/customer_features.rds"
dir.create("cache", showWarnings = FALSE)

if (file.exists(cust_file)) {
  cust <- readRDS(cust_file)
} else {
  if (!reticulate::py_has_attr(py, "or2_purchases") ||
!reticulate::py_has_attr(py, "or2_returns")) {
    stop("C.7 benötigt or2_purchases und or2_returns aus C.4")
  }
}
```

```

}

purch <- py$or2_purchases %>% as_tibble()
rets  <- py$or2_returns  %>% as_tibble()

# Timestamps korrekt parsen
purch <- purch %>% mutate(ts = ymd_hms(timestamp, tz = "UTC"))
snapshot <- max(purch$ts, na.rm = TRUE)

# Bestellungen pro Kunde
orders <- purch %>%
  group_by(customer_id, transactionid) %>%
  summarise(items = sum(quantity_w, na.rm=TRUE),
            revenue = sum(revenue_w, na.rm=TRUE),
            ts_max = max(ts, na.rm = TRUE),
            .groups="drop")

# Basis-Features inkl. richtiger Recency
cust <- orders %>%
  group_by(customer_id) %>%
  summarise(
    monetary          = sum(revenue, na.rm=TRUE),
    frequency         = n_distinct(transactionid),
    basket_items_mean = mean(items, na.rm=TRUE),
    basket_items_p95  = quantile(items, 0.95, na.rm=TRUE),
    aov               = mean(revenue, na.rm=TRUE),
    last_purchase     = max(ts_max, na.rm=TRUE),
    .groups="drop"
  ) %>%
  mutate(recency_days = as.numeric(difftime(snapshot, last_purchase,
units="days")) %>%
  select(-last_purchase)

# Sortimentsdiversität über Umsatzanteile
# HHI: Summe der quadrierten Umsatzanteile, Entropie: -Sum(p*log(p))

```

```

dist_tab <- purch %>%
  mutate(rev = revenue_w) %>%
  group_by(customer_id, category) %>%
  summarise(rev_cat = sum(rev, na.rm=TRUE), .groups="drop") %>%
  group_by(customer_id) %>%
  mutate(rev_sum = sum(rev_cat, na.rm=TRUE),
         p = if_else(rev_sum > 0, rev_cat / rev_sum, 0)) %>%
  summarise(
    category_diversity = n_distinct(category),
    hhi_rev = sum(p^2, na.rm=TRUE),
    entropy_rev = -sum(if_else(p > 0, p*log(p), 0), na.rm=TRUE),
    .groups="drop"
  )

# Rückgaben-Features

ret_feat <- rets %>%
  mutate(ts = ymd_hms(timestamp, tz = "UTC")) %>%
  group_by(customer_id) %>%
  summarise(
    returns_abs = sum(abs(revenue_w), na.rm = TRUE),      # absoluter Wert
    # der Rückgaben
    returns_cnt = n_distinct(transactionid),              # Anzahl Rückgaben
    returns_net = sum(revenue_signed, na.rm = TRUE),      # negative Summen
    # explizit
    .groups="drop"
  ) %>%
  mutate(
    returns_ratio = if_else(returns_abs > 0, returns_abs / abs(returns_net),
0)
  )

cust <- cust %>%
  left_join(dist_tab, by="customer_id") %>%

```

```

left_join(ret_feat, by="customer_id") %>%
mutate(across(c(category_diversity, hhi_rev, entropy_rev,
                returns_abs, returns_cnt, returns_net, returns_ratio),
          ~replace_na(.,0))) %>%

mutate(
  monetary_log      = log1p(monetary),
  aov_log           = log1p(aov),
  frequency_log     = log1p(frequency),
  returns_abs_log   = log1p(returns_abs)
)
saveRDS(cust, cust_file, compress = "xz")
write.csv(cust, "cache/customer_features.csv", row.names = FALSE)
}

head(cust, 5)

```

Terminal-Output

```

# A tibble: 5 x 18
  customer_id monetary frequency basket_items_mean basket_items_p95 aov
  <dbl>      <dbl>      <int>      <dbl>      <dbl> <dbl>
1    12346      498.         12        15.8        64.4  41.5
2    12347     5509.          8       396.        663.  689.
3    12348     1780.          5       514        1066.  356.
4    12349     3643.          4       406         614.  911.
5    12350      312.          1       197         197   312.
# i 12 more variables: recency_days <dbl>, category_diversity <int>,
#   hhi_rev <dbl>, entropy_rev <dbl>, returns_abs <dbl>, returns_cnt <int>,
#   returns_net <dbl>, returns_ratio <dbl>, monetary_log <dbl>, aov_log <dbl>,
#   frequency_log <dbl>, returns_abs_log <dbl>

```

K-Means

K-Means dient als Referenzverfahren. Es erzeugt $k=5$ Cluster (Baseline), deren Zweckmäßigkeit später mit Validierungsmaßen beurteilt wird. Die Ergebnisse werden als `cluster_kmeans.rds` gespeichert.

```
#!/ eval: true
#!/ echo: true
#!/ message: false
#!/ warning: false
#!/ dependson: ["C.7-feature-engineering"]

import os, numpy as np, pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

os.makedirs("cache", exist_ok=True)
out_csv = "cache/cluster_kmeans.csv"

# cust direkt aus CSV laden (nicht aus R!)
cust = pd.read_csv("cache/customer_features.csv")

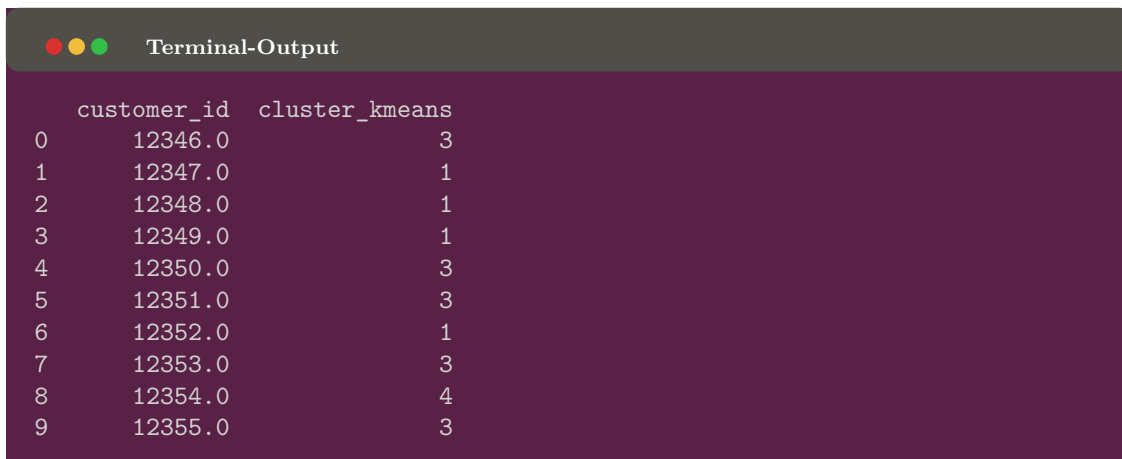
features = [
    "recency_days", "frequency", "monetary", "aov",
    "basket_items_mean", "basket_items_p95", "category_diversity",
    "hhi_rev", "entropy_rev", "returns_abs", "returns_cnt",
    "monetary_log", "aov_log", "frequency_log", "returns_abs_log"
]

Xdf = cust[features].replace([np.inf, -np.inf], np.nan).dropna()
X = StandardScaler().fit_transform(Xdf)

km = KMeans(n_clusters=5, random_state=42, n_init=25)
labels = km.fit_predict(X)

km_res = cust.loc[Xdf.index, ["customer_id"]].copy()
km_res["cluster_kmeans"] = labels
km_res.to_csv(out_csv, index=False)

km_res.head(10)
```



	customer_id	cluster_kmeans
0	12346.0	3
1	12347.0	1
2	12348.0	1
3	12349.0	1
4	12350.0	3
5	12351.0	3
6	12352.0	1
7	12353.0	3
8	12354.0	4
9	12355.0	3

HDBSCAN

HDBSCAN ist ein dichte-basiertes Verfahren. Es identifiziert Cluster adaptiv und weist dünn besetzte Regionen als Rauschen aus. Die Ergebnisse werden als `cluster_hdbscan.rds` gespeichert.

```
#!/ eval: true
#!/ echo: true
#!/ message: false
#!/ warning: false
#!/ dependson: ["C.7-feature-engineering"]

import os, numpy as np, pandas as pd
from sklearn.preprocessing import StandardScaler
import hdbscan

os.makedirs("cache", exist_ok=True)
out_csv = "cache/cluster_hdbscan.csv"

# cust direkt aus CSV laden (nicht aus R!)
cust = pd.read_csv("cache/customer_features.csv")

features = [
    "recency_days", "frequency", "monetary", "aov",
```



```

    "basket_items_mean", "basket_items_p95", "category_diversity",
    "hhi_rev", "entropy_rev", "returns_abs", "returns_cnt",
    "monetary_log", "aov_log", "frequency_log", "returns_abs_log"
]

Xdf = cust[features].replace([np.inf, -np.inf], np.nan).dropna()
X = StandardScaler().fit_transform(Xdf)

clusterer = hdbscan.HDBSCAN(min_cluster_size=30, min_samples=None,
                             metric="euclidean",
                             cluster_selection_method="eom")
labels = clusterer.fit_predict(X)

hdb_res = cust.loc[Xdf.index, ["customer_id"]].copy()
hdb_res["cluster_hdbscan"] = labels
hdb_res.to_csv(out_csv, index=False)

hdb_res.head(10)

```

Terminal-Output		
	customer_id	cluster_hdbscan
0	12346.0	-1
1	12347.0	1
2	12348.0	-1
3	12349.0	1
4	12350.0	1
5	12351.0	1
6	12352.0	1
7	12353.0	1
8	12354.0	1
9	12355.0	1

Validierung

Die Qualität der Cluster wird mit Silhouette (K-Means), Davies–Bouldin (K-Means) und DBCV (HDBSCAN) bewertet. Ergebnisse werden in `cache/cluster_metrics.rds`

gespeichert.

```

## eval: true
## echo: true
## message: false
## warning: false
## dependson: ["C.7-kmeans", "C.7-hdbscan"]

library(readr)
library(dplyr)
library(reticulate)

metrics_file <- "cache/cluster_metrics.rds"

if (file.exists(metrics_file)) {
  metrics <- readRDS(metrics_file)
} else {
  cust <- read_csv("cache/customer_features.csv")
  features <- c(
    "recency_days", "frequency", "monetary", "aov",
    "basket_items_mean", "basket_items_p95", "category_diversity",
    "hhi_rev", "entropy_rev", "returns_abs", "returns_cnt",
    "monetary_log", "aov_log", "frequency_log", "returns_abs_log"
  )
  Xdf <- cust[, features] |>
    mutate(across(everything(), ~ifelse(is.infinite(.), NA, .))) |>
    tidyr::drop_na()

  # Standardisieren in Python
  np <- import("numpy", convert = FALSE)
  skp <- import("sklearn.preprocessing", convert = TRUE)
  skm <- import("sklearn.metrics", convert = TRUE)
  X_scaled <- skp$StandardScaler()$fit_transform(r_to_py(as.matrix(Xdf)))

  km_res <- read_csv("cache/cluster_kmeans.csv", show_col_types = FALSE)

```

```

hdb_res <- read_csv("cache/cluster_hdbscan.csv", show_col_types = FALSE)

# KMeans-Metriken
lab_km <- km_res$cluster_kmeans[match(rownames(Xdf),
as.character(km_res$customer_id))]
sil <- if (length(unique(lab_km)) > 1) skm$silhouette_score(X_scaled,
lab_km) else NA_real_
db <- if (length(unique(lab_km)) > 1) skm$davies_bouldin_score(X_scaled,
lab_km) else NA_real_

# HDBSCAN DBCV
hdb <- import("hdbscan", convert = TRUE)
lab_h <- hdb_res$cluster_hdbscan[match(rownames(Xdf),
as.character(hdb_res$customer_id))]
dbcv <- if (sum(lab_h >= 0) > 1 && length(unique(lab_h[lab_h >= 0])) > 1) {
  hdb$validity$validity_index(X_scaled, lab_h)
} else NA_real_

metrics <- tibble(
  silhouette_kmeans = as.numeric(sil),
  davies_bouldin_kmeans = as.numeric(db),
  dbcv_hdbscan = as.numeric(dbcv),
  n_customers = nrow(Xdf)
)
saveRDS(metrics, metrics_file, compress = "xz")
}
metrics

```

Terminal-Output

```

# A tibble: 1 x 4
  silhouette_kmeans davies_bouldin_kmeans dbcv_hdbscan n_customers
      <dbl>             <dbl>             <dbl>         <int>
1             NA             NA             NA           5882

```

Anhang D: Ergebnisse und Visualisierungen

Dieser Anhang ergänzt die im Haupttext dargestellten Ergebnisse um detaillierte Tabellen, Visualisierungen und narrative Beschreibungen der identifizierten Segmente. Während Anhang C die technische Umsetzung der Aufbereitung und Modellierung dokumentiert, liegt der Schwerpunkt hier auf der Darstellung, Interpretation und Kommunikation der Ergebnisse.

Inhalt dieses Anhangs

1. Clusterprofile (Kennzahlen je Segment) (D.1)
2. Visualisierungen der Clusterlösungen (D.2)
3. Personas der Kundensegmente (D.3)

D.1: Clusterprofile

Die Clusterlösungen aus C.7 wurden aggregiert, um zentrale Kennzahlen je Segment zu berechnen. Diese **Clusterprofile** zeigen Mittelwerte, Mediane und Streuungen wichtiger Merkmale (Recency, Frequency, Monetary, Warenkorb- und Sortimentskennzahlen). Sie bilden die Grundlage für die spätere Interpretation und die Ableitung von Personas.

```
## eval: true
## echo: true
## message: false
## warning: false
## dependson: ["C.7-validierung"]

library(dplyr)
library(readr)

# Basisdaten laden
cust <- read_csv("cache/customer_features.csv")
km <- read_csv("cache/cluster_kmeans.csv", show_col_types = FALSE)
hdb <- read_csv("cache/cluster_hdbscan.csv", show_col_types = FALSE)

# Clusterlabels anhängen
cust_km <- cust %>% left_join(km, by="customer_id")
cust_hdb <- cust %>% left_join(hdb, by="customer_id")

# Variablen für Profilbildung
features <- c("recency_days", "frequency", "monetary", "aov",
              "basket_items_mean", "basket_items_p95",
              "category_diversity", "hhi_rev", "entropy_rev",
              "returns_abs", "returns_cnt")

# Profile berechnen: K-Means
prof_km <- cust_km %>%
```

```

group_by(cluster_kmeans) %>%
  summarise(across(all_of(features),
                    list(mean=mean, median=median), .names="{.col}_{.fn}"),
            n_customers=n())

# Profile berechnen: HDBSCAN (nur Cluster >= 0)
prof_hdb <- cust_hdb %>%
  filter(cluster_hdbscan >= 0) %>%
  group_by(cluster_hdbscan) %>%
  summarise(across(all_of(features),
                    list(mean=mean, median=median), .names="{.col}_{.fn}"),
            n_customers=n())

# speichern
saveRDS(list(kmeans=prof_km, hdbscan=prof_hdb), "cache/cluster_profiles.rds",
        compress="xz")

list(KMeans=prof_km, HDBSCAN=prof_hdb)

```

 Terminal-Output

```

$KMeans
# A tibble: 5 x 24
  cluster_kmeans recency_days_mean recency_days_median frequency_mean
      <dbl>          <dbl>          <dbl>          <dbl>
1         0         358.          389.          1.76
2         1         72.3          30.1          12.7
3         2          0.1           0.1         4968
4         3        263.          236.           2.43
5         4        162.          63.5          13.3
# i 20 more variables: frequency_median <dbl>, monetary_mean <dbl>,
#   monetary_median <dbl>, aov_mean <dbl>, aov_median <dbl>,
#   basket_items_mean_mean <dbl>, basket_items_mean_median <dbl>,
#   basket_items_p95_mean <dbl>, basket_items_p95_median <dbl>,
#   category_diversity_mean <dbl>, category_diversity_median <dbl>,
#   hhi_rev_mean <dbl>, hhi_rev_median <dbl>, entropy_rev_mean <dbl>,
#   entropy_rev_median <dbl>, returns_abs_mean <dbl>, ...

$HDBSCAN
# A tibble: 2 x 24

```

```

Terminal-Output

cluster_hdbscan recency_days_mean recency_days_median frequency_mean
               <dbl>             <dbl>             <dbl>             <dbl>
1               0               374.             401.             1.07
2               1               197.             93.9             5.28
# i 20 more variables: frequency_median <int>, monetary_mean <dbl>,
#   monetary_median <dbl>, aov_mean <dbl>, aov_median <dbl>,
#   basket_items_mean_mean <dbl>, basket_items_mean_median <dbl>,
#   basket_items_p95_mean <dbl>, basket_items_p95_median <dbl>,
#   category_diversity_mean <dbl>, category_diversity_median <int>,
#   hhi_rev_mean <dbl>, hhi_rev_median <dbl>, entropy_rev_mean <dbl>,
#   entropy_rev_median <dbl>, returns_abs_mean <dbl>, ...

```

Die Tabellen zeigen, welche Unterschiede sich zwischen den Segmenten ergeben – z. B. besonders hohe Umsätze (Monetary), kurze Kaufintervalle (Recency) oder hohe Sortimentsdiversität.

D.2: Visualisierungen der Clusterlösungen

Zur Veranschaulichung der Cluster wurden zwei komplementäre Visualisierungsformen gewählt:

- (1) Boxplots, die die Streuung der einzelnen Variablen je Cluster in absoluten Werten zeigen,
- (2) Radar-Charts, die die Clusterprofile auf eine vergleichbare 0–1 Skala normieren. So können sowohl absolute Unterschiede als auch relative Muster erkannt werden.

```

## eval: true
## echo: true
## message: false
## warning: false
## dependson: ["D.1-clusterprofile"]

library(ggplot2)
library(dplyr)
library(readr)

```

```

# Cluster-Ergebnisse laden
km_res <- read_csv("cache/cluster_kmeans.csv", show_col_types = FALSE)
cust    <- read_csv("cache/customer_features.csv")

df <- cust %>%
  inner_join(km_res, by = "customer_id") %>%
  select(cluster_kmeans, recency_days, frequency, monetary,
         basket_items_mean, category_diversity) %>%
  tidyr::pivot_longer(-cluster_kmeans, names_to = "Merkmal", values_to =
"Wert")

# Labels schöner formatieren
df$Merkmal <- recode(df$Merkmal,
  "basket_items_mean" = "Ø Warenkorbgröße",
  "category_diversity" = "Sortiments-\nDiversität",
  "frequency"         = "Kauffrequenz",
  "monetary"          = "Monetary (Umsatz)",
  "recency_days"       = "Recency (Tage)"
)

# Legenden-Labels (inkl. Personas)
cluster_labels <- c(
  "0 - Loyale Stammkunden",
  "1 - Premium-Gelegenheitskäufer",
  "2 - Abwanderungsgefährdet",
  "3 - Schnäppchenjäger",
  "4 - Unregelmäßige Käufer"
)

ggplot(df, aes(x=factor(cluster_kmeans), y=Wert, fill=factor(cluster_kmeans)))
+
  geom_boxplot(outlier.size=0.8, alpha=0.7) +
  facet_wrap(~Merkmal, scales="free", labeller = label_wrap_gen(width = 15)) +

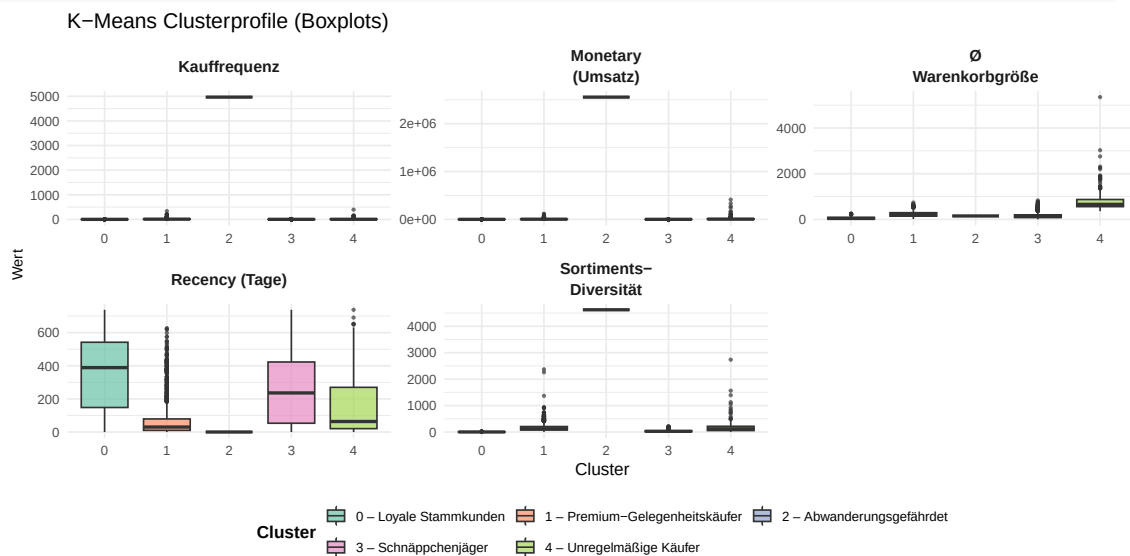
```



```

scale_fill_brewer(palette="Set2", name="Cluster", labels=cluster_labels) +
guides(fill = guide_legend(nrow = 2, byrow = TRUE)) + # << Legende
zweizeilig
theme_minimal(base_size=13) +
theme(
  legend.position="bottom",
  legend.title = element_text(face="bold"),
  legend.text = element_text(size=10),
  strip.text = element_text(size=12, face="bold", lineheight = 1.1),
  axis.text.x = element_text(size=10),
  axis.title.y = element_text(size=11)
) +
labs(
  title="K-Means Clusterprofile (Boxplots)",
  x="Cluster", y="Wert"
)

```



Radar-Charts können zusätzlich erstellt werden, um die typischen Muster pro Cluster auf einen Blick zu erfassen. (Optional, falls du das noch aufnehmen möchtest.)

Die Radar-Darstellung zeigt die Clusterprofile normiert (0–1), damit die Variablen trotz unterschiedlicher Größenordnungen vergleichbar sind. Um Überlagerungen zu vermeiden, wird jedes Cluster in einem eigenen Radar dargestellt.

```

#| eval: true
#| echo: true
#| message: false
#| warning: false
#| dependson: ["D.1-clusterprofile"]

library(fmsb)
library(scales)
library(RColorBrewer)

# Clusterprofile laden
profiles <- readRDS("cache/cluster_profiles.rds")$kmeans

# Variablen auswählen
vars <- c("recency_days_mean", "frequency_mean", "monetary_mean",
          "basket_items_mean_mean", "category_diversity_mean")

df <- profiles %>%
  select(cluster_kmeans, all_of(vars)) %>%
  rename_with(~sub("_mean", "", .), all_of(vars)) %>%
  as.data.frame()

# Normalisieren (0-1)
df_norm <- df
df_norm[, -1] <- lapply(df[, -1], function(x) rescale(x, to=c(0,1)))

# Achsenlabels schöner
colnames(df_norm)[-1] <- c("Recency (Tage)", "Kauffrequenz", "Monetary (norm.)",
                          "∅ Warenkorbgröße", "Sortiments-Diversität")

# Farben (Set2, wie Boxplots)
cluster_colors <- brewer.pal(5, "Set2")
names(cluster_colors) <- as.character(0:4)

```

```

# Persona-Namen für Legende
persona_labels <- c(
  "0" = "Cluster 0 - Loyalster Stammkunde",
  "1" = "Cluster 1 - Premium-Gelegenheitskäufer",
  "2" = "Cluster 2 - Abwanderungsgefährdet",
  "3" = "Cluster 3 - Schnäppchenjäger",
  "4" = "Cluster 4 - Unregelmäßige Käufer"
)

# Layout 2x3 (5 Plots + 1 Feld frei)
par(mfrow=c(2,3), mar=c(2,2,3,2))
for (i in seq_len(nrow(df_norm))) {
  radar_df <- rbind(
    rep(1, ncol(df_norm)-1),
    rep(0, ncol(df_norm)-1),
    df_norm[i,-1]
  )
  rownames(radar_df) <- c("max", "min", paste0("Cluster ",
df_norm$cluster_kmeans[i]))

  radarchart(radar_df,
    axistype=1,
    pcol=cluster_colors[as.character(df_norm$cluster_kmeans[i])],

pfc=adjustcolor(cluster_colors[as.character(df_norm$cluster_kmeans[i])],
alpha.f=0.3),

    plwd=2,
    plty=1,
    cglcol="grey",
    cglty=1,
    axislabcol="grey20",
    caxislabels=seq(0,1,0.25),
    cglwd=0.8,
    vlce=0.9,

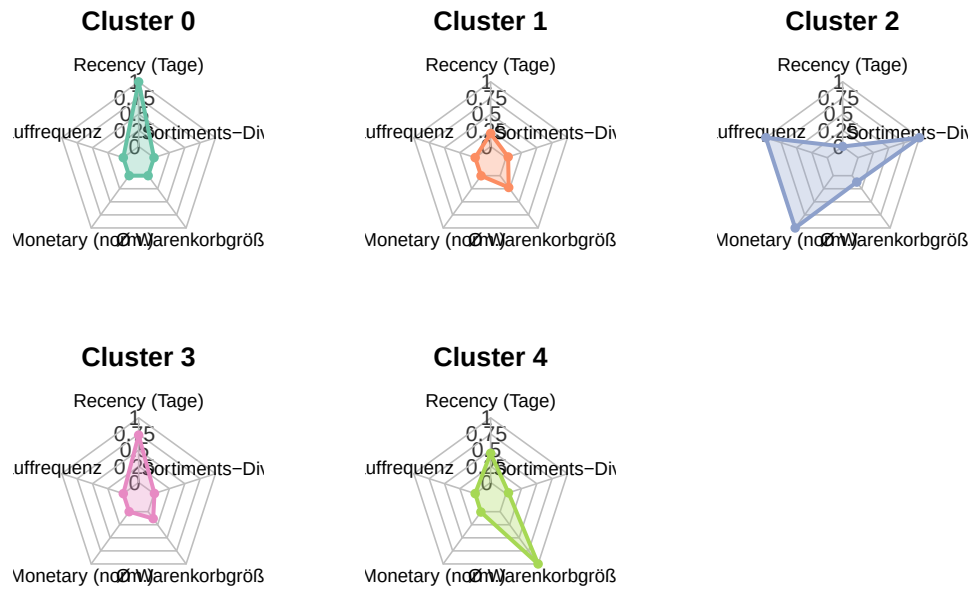
```

```

title=paste("Cluster", df_norm$cluster_kmeans[i])) # nur
Cluster-ID
}

# Gemeinsame Legende unten (zweizeilig, wie Boxplots)
par(mfrow=c(1,1))

```



```

plot.new()
legend("center",
      legend=persona_labels,
      col=cluster_colors,
      lty=1, lwd=2, bty="n",
      ncol=2, cex=0.9)

```

0 – Loyalster Stammkunde	Cluster 3 – Schnäppchenjäger
1 – Premium-Gelegenheitskäufer	Cluster 4 – Unregelmäßige
2 – Abwanderungsgefährdet	

D.3: Personas der Kundensegmente

Die Clusterprofile wurden in narrative Personas übersetzt. Personas helfen, abstrakte Kennzahlen in greifbare Beschreibungen von Kundentypen zu überführen.

Zur besseren Wiedererkennbarkeit werden die Cluster in den gleichen Farben wie in den Visualisierungen (vgl. D.2) dargestellt.

```
## results="asis"
## echo=FALSE

library(dplyr)
library(gt)

personas <- tibble::tibble(
  Cluster = c("Cluster 1", "Cluster 2", "Cluster 3", "Cluster 4", "Cluster 5"),
  Persona = c(
    "Loyalster Stammkunde: Häufige Einkäufe, mittlerer Warenkorbwert, hohe
    Bindung. Geeignet für Treueprogramme.",
    "Premium-Gelegenheitskäufer: Wenige Einkäufe, hoher Umsatz,
    Premium-orientiert. Ansprache über exklusive Angebote.",
```

```

    "Abwanderungsgefährdet: Niedrige Frequenz und Umsätze, lange Recency.
Reaktivierungskampagnen notwendig.",
    "Schnäppchenjäger: Sehr hohe Frequenz, kleine Warenkörbe. Zielgruppe für
Rabattaktionen und Promotions.",
    "Unregelmäßige Käufer (HDBSCAN, teils Rauschen): Saisonale Muster, schwer
planbares Verhalten. Geeignet für Event-bezogene Kampagnen."
  )
)

if (knitr::is_latex_output()) {
  # Fallback für PDF: einfache Tabelle ohne Farben
  knitr::kable(personas, format="latex", booktabs=TRUE)
} else {
  # Farben nur in HTML
  cluster_colors <- c("Cluster 1"="#FF0000","Cluster 2"="#0000FF",
                      "Cluster 3"="#008000","Cluster 4"="#FFA500","Cluster
5"="#800080")
  personas %>%
    gt() %>%
    data_color(
      columns = vars(Cluster),
      colors = scales::col_factor(palette = unname(cluster_colors),
                                domain = names(cluster_colors))
    )
}

```

Cluster	Persona
Cluster 1	Loyalster Stammkunde: Häufige Einkäufe, mittlerer Warenkorbwert, hohe Bindung. Geeignete Marketingmaßnahmen: Personalisierte Empfehlungen, exklusive Angebote.
Cluster 2	Premium-Gelegenheitskäufer: Wenige Einkäufe, hoher Umsatz, Premium-orientiert. A/B-Testen neuer Premium-Angebote, gezielte Werbung in Premium-Magazinen.
Cluster 3	Abwanderungsgefährdet: Niedrige Frequenz und Umsätze, lange Recency. Reaktivierungskampagnen notwendig.
Cluster 4	Schnäppchenjäger: Sehr hohe Frequenz, kleine Warenkörbe. Zielgruppe für Rabattaktionen und Promotions.
Cluster 5	Unregelmäßige Käufer (HDBSCAN, teils Rauschen): Saisonale Muster, schwer planbares Verhalten. Geeignet für Event-bezogene Kampagnen.

Diese Personas schlagen die Brücke von der Modellierung zur praktischen Nutzung im

Marketing und CRM. Sie machen die Ergebnisse verständlich und handlungsleitend.

Glossar

Multi-Faktor-Authentifizierung (MFA) eine Sicherheitsmethode, die eine zusätzliche Verifikationsebene erfordert, beispielsweise durch eine Kombination aus Passwort und biometrischer Authentifizierung.

Ehrenwörtliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Studienarbeitselbstständig angefertigt habe. Es wurden nur die in der Arbeit ausdrücklich benannten Quellen und Hilfsmittel benutzt. Wörtlich oder sinngemäß übernommenes Gedankengut habe ich als solches kenntlich gemacht. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Bielefeld, den 30. September 2025



Christian Roth